



Project Campus TM

“A Unified Academic Social Platform”

Project By

Ayham Al Sayyed

Ibrahim Al Sharif

Laith Abu Arrah

2025 – 2026

Table of Contents

Abstract.....	3
Introduction.....	3
Related Work	4
Problem Statement.....	4
Project Objectives	5
• Objective:.....	5
• Sub-Objectives:.....	5
Project Benefits.....	6
Project Methodology	6
Project Constraints	11
Project Implementation Options.....	14
The Proposed System	15
System Requirements	17
Functional Requirements:.....	17
• Main Functional Requirements:.....	17
• Advanced Functional Requirements:.....	18
Non-Functional Requirements:.....	20
Feasibility Study	21
• Economic Feasibility:	21
• Technical Feasibility:.....	24
• Legal Feasibility:.....	24
Project Added Values	25
Project Management	26
Diagrams.....	28
• Use Case Diagram	28
• Flowchart Diagram	32
• Sequence Diagram	41
• Entity Relationship Diagram	48
User Interface	50
Coding Programming Languages	55
Conclusion	56
Future Work	57

Abstract

Project Campus is a unified digital platform designed to centralize academic communication and reduce reliance on informal channels such as WhatsApp and Facebook. It provides a structured environment for accessing official announcements, academic updates, and university information without fragmentation or misinformation. By connecting students across multiple universities through organized pages, instructor profiles, and interactive communities, the platform enhances communication, collaboration, and overall student engagement.

Introduction

Effective communication plays a vital role in the academic environment, influencing how students receive information and interact within their universities. Many students rely on informal platforms such as WhatsApp and Facebook for academic updates; however, these channels often lack structure, reliability, and academic focus.

Project Campus addresses this issue by providing a unified digital platform that centralizes academic communication. The system offers official announcements, university-specific pages, faculty profiles, and organized spaces for academic and social interaction, enabling more efficient and reliable engagement.

By reducing information fragmentation and improving accessibility, Project Campus aims to strengthen communication between students, instructors, and universities, ultimately enhancing student engagement and the overall academic experience.

Related Work

Platform	Purpose / Features	Limitations	Relevance to Project Campus
 WhatsApp	Instant messaging, group chats	Unstructured, no verification, poor archiving	Fast communication but lacks academic structure
 Facebook	Groups, events, peer interaction	Misinformation, algorithm hides updates	Social engagement but unreliable academically
 Instagram	Visual content, event highlights	No structured academic content, image-focused	Good engagement, weak academic communication
 Reddit	Discussions, Q&A, knowledge sharing	Anonymous, not university-specific	Good discussion, no official verification
 Moodle	Course management, announcements	Limited social interaction	Reliable academically, weak community aspect
 Project Campus	Unified academic communication platform	Single-university initially	Integrated, verified, scalable solution

Problem Statement

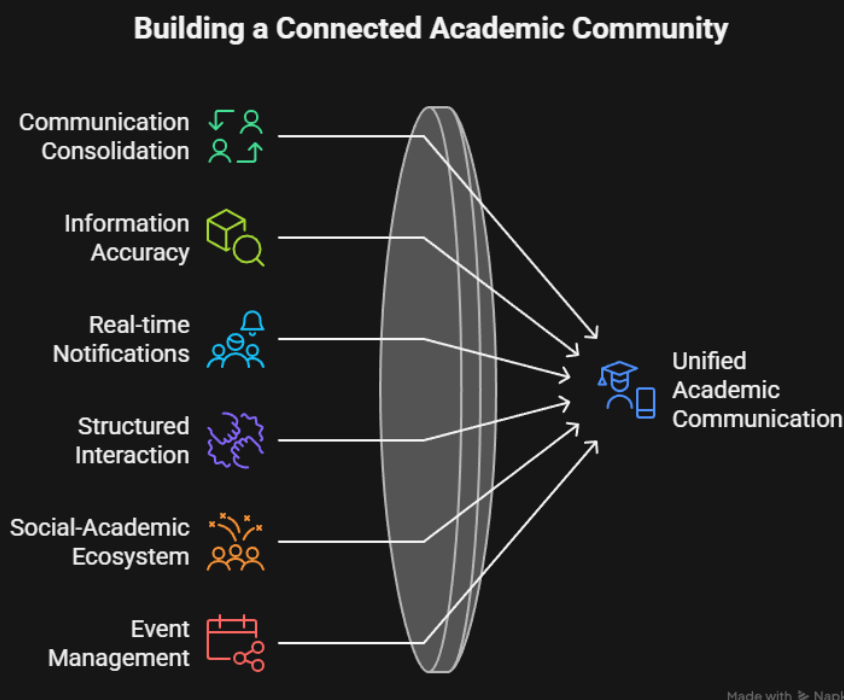
The absence of a unified communication system creates several issues, including misinformation, delayed updates, and the fragmentation of important content across multiple platforms. Furthermore, students often struggle to identify and connect with instructors due to the lack of centralized academic profiles. This fragmented environment not only affects the flow of information but also limits the ability of students from different universities to exchange experiences, collaborate, and build academic networks in a structured, meaningful way.

Project Objectives

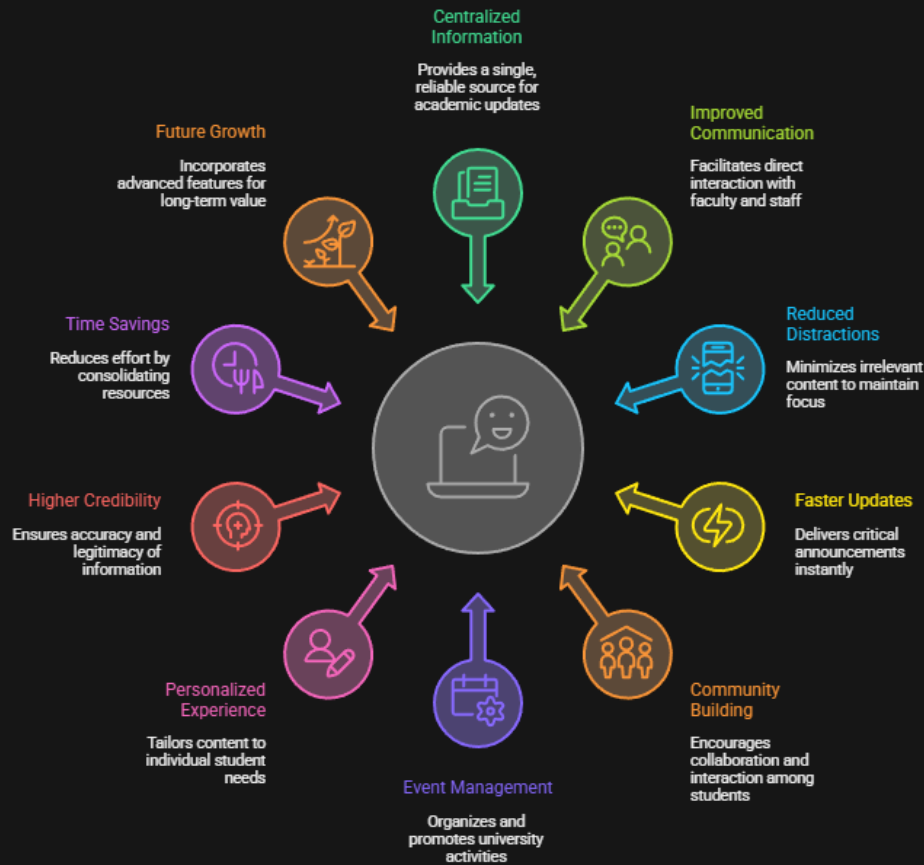
- **Objective:**

- **Establish a Unified and Reliable Academic Communication Platform:** to design and implement a centralized digital platform that replaces fragmented communication channels and ensures accurate, verified, and timely academic information delivery for university students.
- **Enhance Academic and Social Interaction Within the University Community:** to improve communication, collaboration, and engagement between students, faculty, and academic communities through structured and organized digital interaction.
- **Deliver a Secure, Scalable, and Future-Ready Digital System:** to develop a modern, user-friendly platform supported by a robust technical architecture that ensures security, performance, and long-term expandability.

- **Sub-Objectives:**



Project Benefits



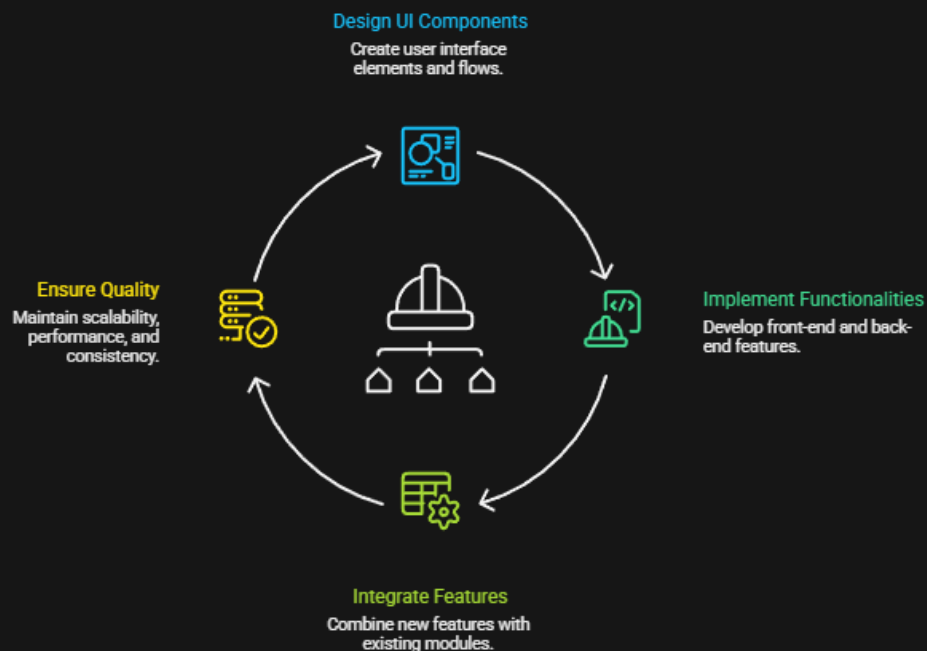
Made with  Napkin

Project Methodology

Project Campus is developed using the “Agile methodology”, which provides flexibility, iterative development, and continuous enhancement throughout the project lifecycle. This approach is well-suited for a dynamic platform that requires frequent updates, user feedback, and the gradual introduction of new features.

- **Requirements gathering:** the project begins with collecting and analyzing user needs based on current academic communication challenges. Functional and non-functional requirements are identified through observation, research, and problem analysis.
- **Backlog:** these requirements are recorded in a **product backlog**, which contains all features, improvements, and tasks prioritized according to importance and impact. This ensures the development process starts with a clear understanding of what the system must achieve.
- **Planning:** the project is divided into repeating development cycles known as “**sprints**”. During sprint planning, the team selects a set of backlog items to be implemented in the upcoming cycle. The selection is based on priority, technical feasibility, and team capacity. Each sprint typically focuses on delivering a complete, functional part of the system.
- **Incremental development:** during each sprint, the team follows an iterative process to build and refine the selected features.

Incremental Development Cycle



Made with  Napkin

- **Continuous testing & Quality assurance:** testing is integrated into every phase of development rather than being isolated at the end. The team performs:



Made with  Napkin

Continuous testing ensures early detection of issues, improves reliability, and supports the rapid pace of Agile development.

- **Sprint review and user feedback:** at the end of each sprint, the team evaluates the completed work in a sprint review. Functional features are demonstrated to gather early feedback from potential users such as students or academic staff. This feedback is then analyzed and used to update the product backlog. By integrating changes early, the platform evolves based on actual user expectations rather than assumptions.

- **Sprint Retrospective and Process Improvement:** following every sprint, the team conducts a retrospective meeting to reflect on the development process. The objectives include:



This practice ensures continuous enhancement of team productivity, task organization, and communication.

- **Deployment and Incremental Releases:** instead of waiting for full system completion, Agile supports delivering **incremental releases**. Once features pass testing and review, they are deployed to a live or semi-live environment. This allows students and universities to benefit from essential functionality early, while additional features continue to be developed in future sprints.
- **Maintenance and Future Enhancements:** after deployment, the platform enters a phase of continuous maintenance. This includes:

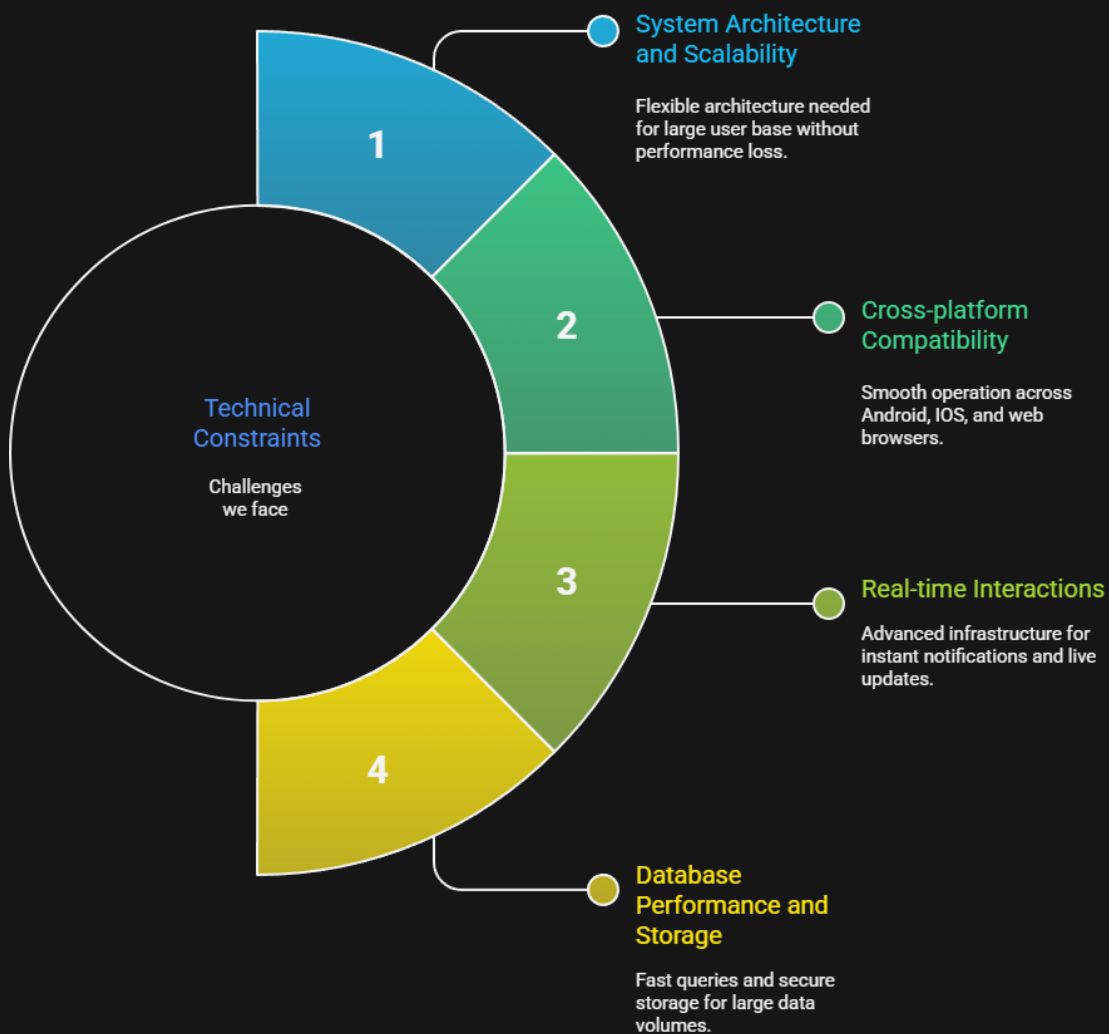


Made with  Napkin

This ensures Project Campus remains stable, scalable, and aligned with evolving user needs and academic requirements.

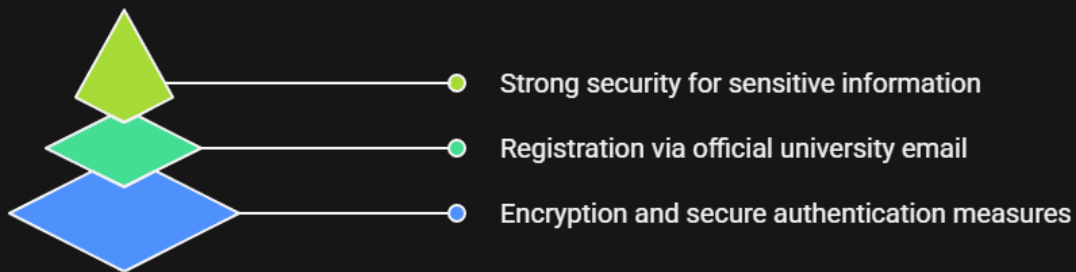
Project Constraints

- **Technical constraints:** these constraints limit how the system can be built and maintained:



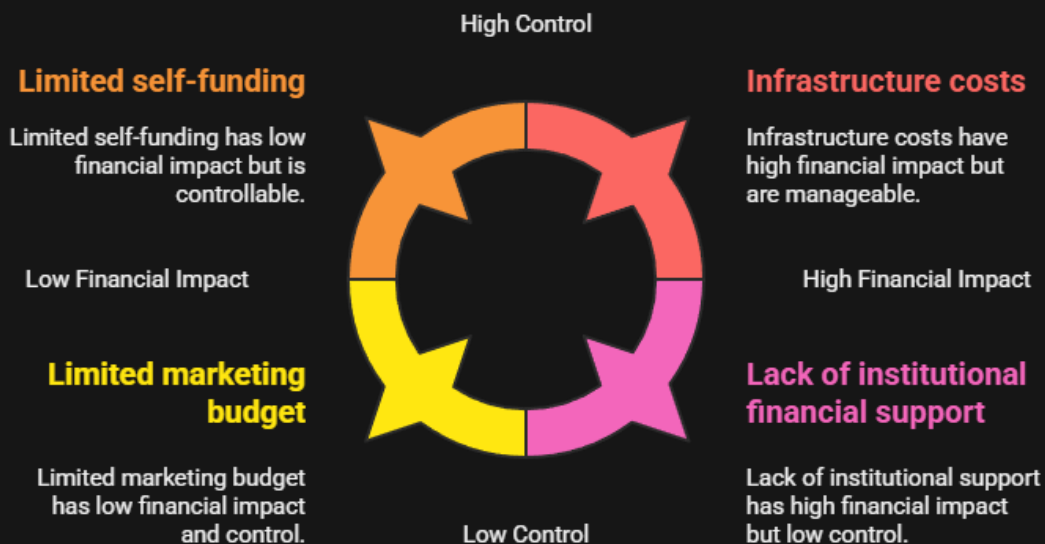
Made with  Napkin

- **Security constraints:** since the system handles academic and personal information, strong security is required.



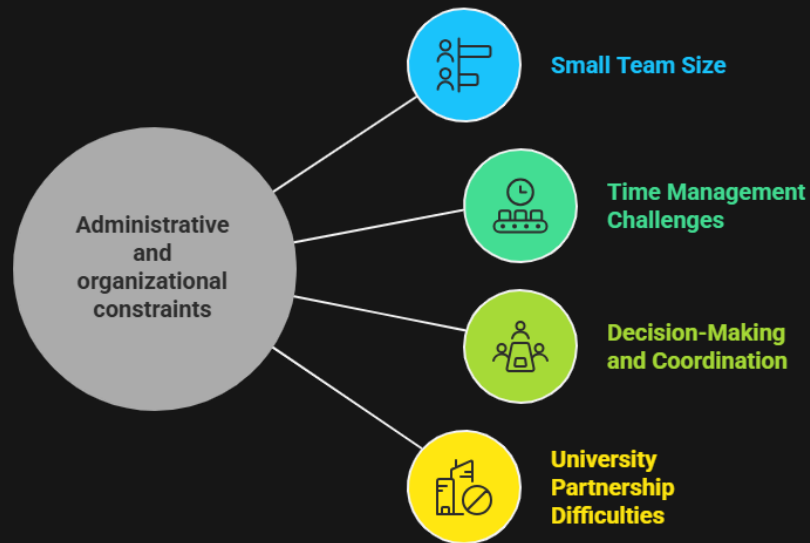
Made with Napkin

- **Financial constraints:** these constraints affect the speed of development and the tools available:



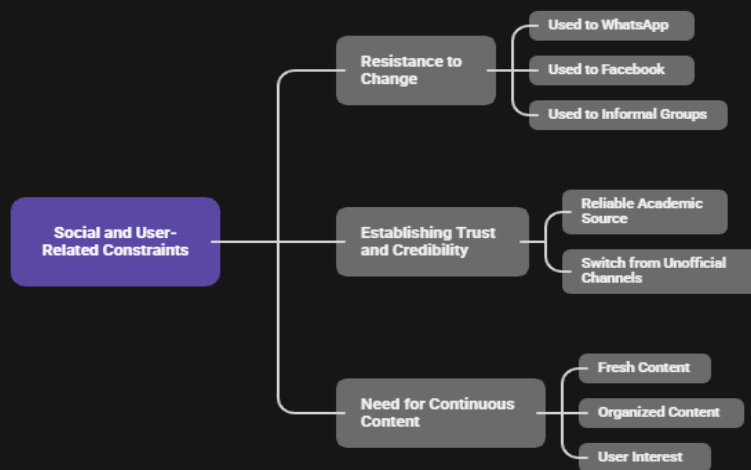
Made with Napkin

- **Administrative and organizational constraints:**



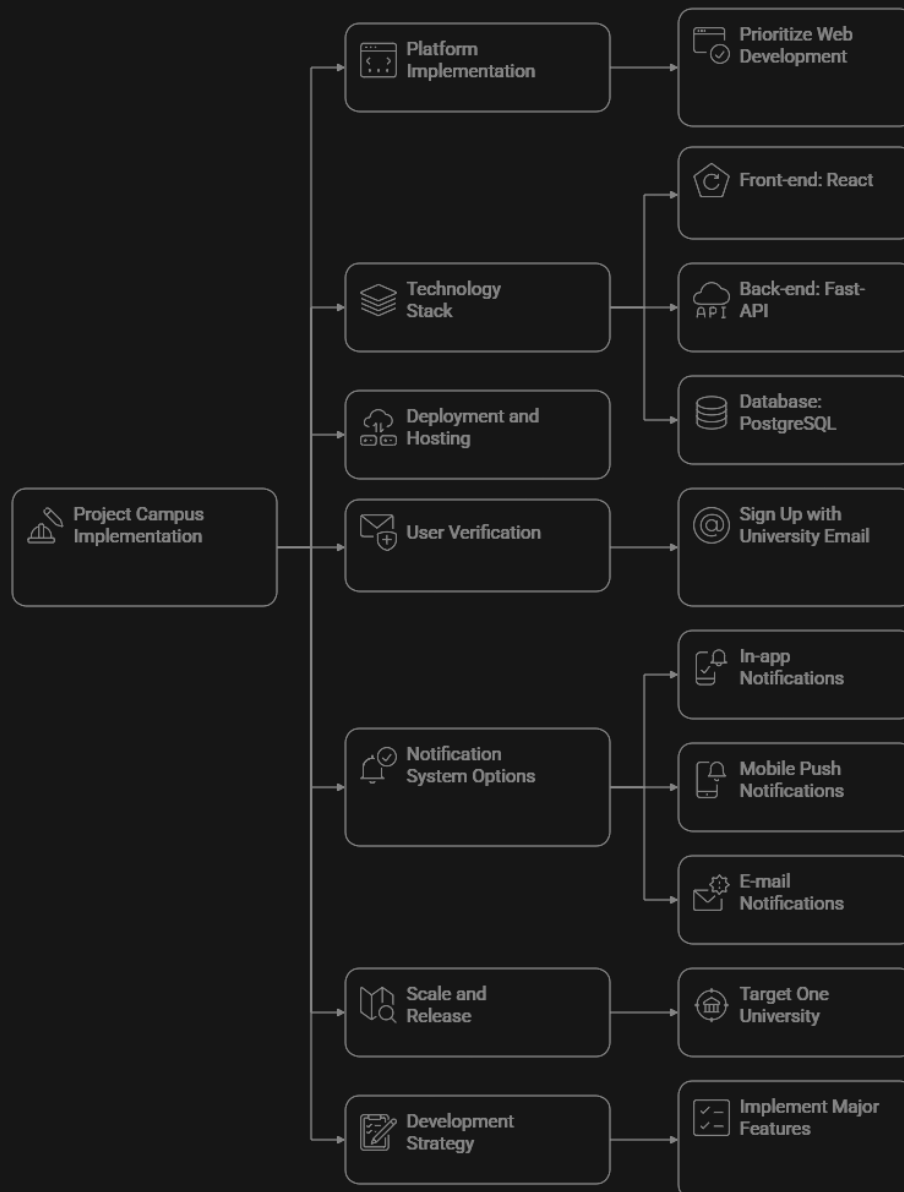
Made with Napkin

- **Social and user-related constraints:**



Made with Napkin

Project Implementation Options



The Proposed System

Project Campus is a unified digital platform designed to centralize academic communication and reduce information fragmentation within university communities. It provides a structured and reliable environment that enables students, instructors, and academic institutions to exchange information efficiently, addressing the limitations of informal communication channels such as WhatsApp groups and Facebook pages.

The platform offers a modern interface that allows access to university announcements, academic updates, faculty profiles, and community discussions from a single environment. Verified university accounts ensure content authenticity, while personalized profiles and university-specific pages deliver relevant information based on enrolled institutions and courses.

Project Campus supports interactive communities for academic collaboration and social engagement, along with streamlined content sharing for text, images, and links. A dynamic notification system delivers important updates through in-app alerts, email, and mobile push notifications.

Built on a scalable architecture using React, FastAPI, and PostgreSQL, the platform is designed to initially support a single university and expand to multiple institutions, with future enhancements such as AI-assisted academic tools and engagement features. By improving communication efficiency and collaboration, Project Campus aims to enhance the overall student experience.

Main System Functions:

- User registration and authentication
- University and faculty profile management
- Academic announcements and notifications
- Community discussions and content sharing
- Personalized feeds and updates

Actors and Functions:

This section presents the main system actors and their associated functional responsibilities within Project Campus. The table below clarifies role-based access to system features, ensuring a clear separation of privileges between students, instructors, and Official Pages. This mapping supports system security, accountability, and consistency with functional requirements.

<i>System Function</i>	<i>Student</i>	<i>Instructor</i>	<i>Page</i>
User Registration & Authentication	Yes	Yes	Yes
Profile Creation & Management	Yes	Yes	Yes
View Faculty Directory	Yes	Yes	Yes
View Official Announcements	Yes	Yes	Yes
Publish Official Announcements	No	No	Yes
Create Posts	Yes	Yes	Yes
Interact With posts	Yes	Yes	Yes
Join Communities	Yes	Yes	No
Create Communities	Yes	Yes	Yes
Create & Manage Events	No	No	Yes
Private Messaging	Yes	Yes	Yes
Academic Group Chats	Yes	Yes	No
Receive Notifications	Yes	Yes	Yes
Manage Notification Preferences	Yes	Yes	Yes
Report Content	Yes	Yes	Yes
Content Moderation	No	No	Yes
User Suspension / Banning	No	No	Yes
Official Page Management	No	No	Yes

The following notes clarify specific constraints and behaviors related to role-based functionality:

- Official Pages can create posts only on their own pages.
- Official Pages may participate in communities only upon approval and within relevant topics.
- Academic group chats are automatically created by the system and managed by instructors.
- Official Pages may be discovered through search functionality and personalized home feed recommendations based on user interests and joined communities.

System Requirements

Functional Requirements:

- **Main Functional Requirements:**
 - **User registration and authentication:** The system shall provide a secure registration and login mechanism that allows students, instructors, and university administrators to access the platform. Account creation shall require the use of official academic email addresses to ensure identity verification and prevent impersonation. The system shall also support secure session management and password recovery functionality.
 - **User profile management:** The system shall allow users to create, view, and update personal profiles based on their assigned roles, including student and instructor. Each user profile shall be associated with a specific university to enable role-based access control and personalized content delivery. In addition, the system shall provide official **University pages** that represent academic institutions. These pages shall serve as information hubs and may be followed by users to receive official announcements, updates, and events published by the institution.

- **Home feed and content interaction:** The system shall provide a centralized home feed that displays posts, official announcements, and updates relevant to the user. Users shall be able to create posts containing text, images, or links, and interact with content through reactions, comments, and sharing features.
 - **Community management:** The system shall enable users to browse, join, and participate in communities based on academic or personal interests. Communities may be configured as public or private, and users shall be able to engage in discussions and share content within these communities according to access permissions.
 - **Event management:** The system shall allow **authorized official entities**, including universities, academic institutions, cooperating organizations, and approved student-related venues (such as libraries, cafés, or learning spaces), to create and publish academic and social events. Event details, including date, time, location, and description, shall be displayed to users. Users shall be able to follow events in order to receive notifications and updates related to them.
 - **University pages:** The system shall provide dedicated pages for universities that serve as official information hubs. These pages shall display verified announcements, news, events, and institutional updates published by authorized administrators.
 - **Faculty and staff directory:** The system shall include a directory of instructors and academic staff associated with each university. Users shall be able to view instructor profiles containing relevant academic and contact information.
 - **Search functionality:** The system shall provide a unified search feature that allows users to search for users, communities, events, and content across the platform.
- **Advanced Functional Requirements:**
 - **Messaging and chat system:** The system shall support private messaging between users to enable direct communication. In addition, course-related chat groups shall be automatically created by the system and managed by instructors, with enrolled students being automatically added to facilitate academic discussions and coordination. Private

messaging with Official Pages is intended for inquiries and institutional communication and is subject to moderation.

- **Notification management:** The system shall provide a notification mechanism to inform users of important events, announcements, messages, and interactions. Notifications shall be delivered through in-app alerts, email messages, and mobile push notifications.
- **Role-based access control:** The system shall support multiple user roles, including students, instructors, and university administrators. Access to administrative and moderation functionalities shall be restricted based on the user's assigned role.
- **Content management and editing:** The system shall allow authorized university administrators to edit or remove published posts and events when necessary, ensuring content accuracy and relevance.
- **Content moderation and reporting:** The system shall provide mechanisms for users to report inappropriate or abusive content. Reported content shall be reviewed by authorized university administrators, who may take disciplinary actions such as content removal, user suspension, or banning in coordination with the relevant academic institution.
- **Content verification and authenticity:** The system shall distinguish official university content from regular user-generated content. Only verified university accounts shall be permitted to publish official announcements.
- **Role-based content publishing:** The system shall allow instructors to publish academic-related content with a clear role identifier. Instructor-generated content may be visually distinguished or prioritized to enhance clarity and credibility.
- **Personalization:** The system shall personalize displayed content based on the user's university affiliation, joined communities, followed events, and enrolled courses.
- **Multi-language support:** The system shall support multiple languages to accommodate diverse users. The initial implementation shall support both English and Arabic, allowing users to switch languages through the interface.

Non-Functional Requirements:

- **Usability:** The system shall provide an intuitive and user-friendly interface suitable for students, instructors, and administrators with varying levels of technical experience. Navigation between personal profiles, communities, events, chats, and official pages shall be clear and consistent to ensure efficient user interaction.
- **Performance:** The system shall deliver responsive performance, supporting smooth interactions such as content browsing, messaging, and notifications. The platform shall be capable of supporting thousands of users per university without significant performance degradation.
- **Scalability:** The system shall be designed to initially support a single university environment while allowing future expansion to multiple universities and affiliated institutions. The architecture shall accommodate growth in users, communities, events, and data volume without requiring major structural changes.
- **Availability and reliability:** The system shall aim for continuous availability on a 24/7 basis. During the initial launch phase, a best-effort availability model may be applied. The system shall handle failures gracefully and preserve stored data without loss.
- **Security and privacy:** The system shall protect user data and institutional content through secure authentication and authorization mechanisms. Private chats and private communities shall be accessible only to participating users. Administrative access to private content shall be restricted and permitted only in cases of reported violations or moderation requirements.
- **Data integrity and retention:** The system shall preserve user-generated content, including posts, chats, and events, to maintain academic records and communication history. Stored data shall remain consistent and protected from unauthorized modification or deletion.
- **Accountability and traceability:** The system shall maintain logs of critical actions, including content publishing, moderation activities, and administrative operations. These logs shall enable authorized administrators to trace actions to verified users or official entities when required.

- **Maintainability:** The system shall be developed using modular and maintainable components to facilitate system updates, debugging, and future enhancements without disrupting existing functionality.
- **Language support:** The implementation shall support English and Arabic, allowing users to switch languages seamlessly through the interface.
- **Institutional compliance:** The system shall operate in alignment with general university policies and codes of conduct. Platform rules, moderation actions, and disciplinary measures shall support institutional standards for academic behavior and community interaction.

Feasibility Study

- **Economic Feasibility:**

Project Campus is economically feasible as it is developed using open-source technologies and cost-efficient development practices. The system relies on React, FastAPI, and PostgreSQL, eliminating licensing costs, while development is handled by the project team, avoiding external labor expenses.

Initial deployment targets a single university and can be hosted on low-cost cloud infrastructure under a best-effort availability model, keeping operational costs minimal. The system architecture supports gradual scaling as usage grows, preventing the need for large upfront investments. Maintenance costs are further reduced through a modular design that allows incremental updates and fixes without extensive redesign.

Overall, Project Campus can be developed and operated with minimal financial resources while delivering significant value through improved academic communication and student engagement.

Development Cost (One-Time)

Cost Item	Description	Estimated Cost
Hardware	Laptops & testing devices	\$1,000
Software tools	Design tools (mostly open source)	\$0
UI/UX design	Icons, Assets, Mockups	\$100
Database setup	Initial Database configuration	\$0
Testing & debugging	Manual testing validation	\$0
Domain registration	Domain name (1 year)	\$20
Total development cost		\$1,120

Operational Cost (Annual)

Cost Item	Description	Estimated Cost
Cloud hosting	Server hosting and infrastructure	\$300
Database Hosting	Cloud database service	\$200
Storage	Media and backup storage	\$150
Maintenance	Bug fixes and minor updates	\$250
Security & monitoring	Basic security and monitoring tools	\$150
Total Operational cost (Annual)		\$1,050

Total Estimated Cost

Cost Item	Estimated Cost
Development Cost (One Time)	\$1,120
Operational Cost (1 year)	\$1,050
Total Operational cost (Annual)	\$2,170

Revenue Sources

Revenue Source	Description	Estimated Annual Revenue
University Subscription	Annual fee paid by university	\$4,000
Sponsored Events	Promoted Events	\$1,750
Advertisements	Advertising Services	\$2,450
Total Estimated Revenue (Annual)		\$8,200

Cost-Revenue Comparison

Item	Amount
Total First-Year Cost	\$2,170
Total Annual Revenue	\$8,200
Estimated Net Benefit	\$7,030

- **Technical Feasibility:**

Project Campus is technically feasible as it is designed using a modern, widely adopted technology stack that supports scalability, performance, and maintainability. The system uses React for the frontend to build interactive and responsive user interfaces, FastAPI for the backend to handle high-performance and concurrent requests, and PostgreSQL for reliable and scalable data storage.

The architecture follows a modular design that separates frontend, backend, and database layers, improving maintainability and allowing independent updates and future integrations. The platform can be deployed on standard cloud-based infrastructure and initially targets a single university, with the ability to scale gradually as usage increases.

Based on the selected technologies and the development team's technical expertise, the system can be successfully developed, deployed, and maintained, making Project Campus technically feasible.

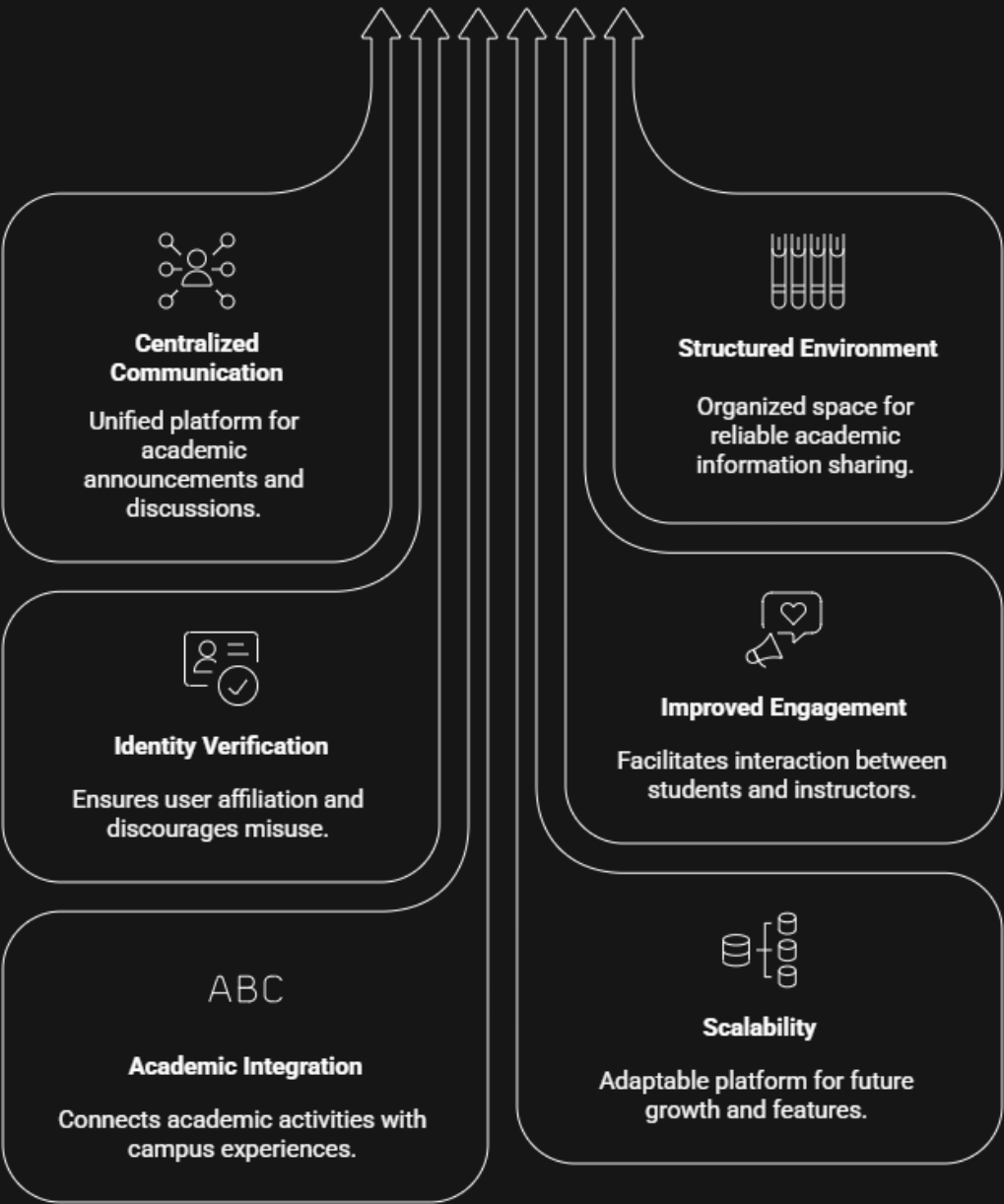
- **Legal Feasibility:**

Project Campus is legally feasible as it is designed to operate within academic and institutional regulations while protecting user rights and privacy. User registration requires official academic email addresses, ensuring verified identities and reducing impersonation and misuse. The platform includes moderation and reporting mechanisms to handle inappropriate content in coordination with relevant academic authorities.

User privacy is maintained by restricting access to private chats and communities to participating members only, with administrative access granted solely for moderation or reported violations. The system avoids collecting unnecessary personal data and securely stores required information for platform operation, moderation, and institutional accountability.

By aligning with university policies, codes of conduct, and responsible data usage practices, Project Campus supports lawful and ethical operation within academic environments, making it legally feasible for academic use and potential future deployment.

Project Added Values



Made with  Napkin

Project Management

The development of **Project Campus** follows a structured project management approach to ensure timely delivery, effective coordination, and alignment with project objectives. The project is managed using an **Agile-based development methodology**, allowing flexibility, continuous improvement, and incremental progress throughout the development lifecycle.

- **Project Planning and Organization:** The project is divided into well-defined phases, including requirements analysis, system design, implementation, testing, and documentation. Each phase is planned with specific goals and deliverables to ensure steady progress and clear evaluation of outcomes. Regular reviews are conducted to assess progress, identify issues, and adjust plans when necessary.
- **Development Methodology:** An Agile approach is adopted to support iterative development and continuous feedback. System features are developed incrementally, starting with core functionalities such as user authentication, profiles, and university pages, followed by advanced features including communities, events, messaging, and moderation mechanisms. This approach allows early identification of design or implementation challenges and supports gradual system refinement.
- **Roles and Responsibilities:** Project responsibilities are distributed among team members based on their technical skills and assigned tasks. Typical responsibilities include frontend development, backend development, database design, UI/UX design, documentation, and testing. Clear role distribution improves coordination and reduces development bottlenecks while encouraging collaborative problem-solving.
- **Schedule and Milestones:** The project schedule is organized around key milestones, such as completion of system requirements, finalization of conceptual designs, implementation of core features, integration of advanced functionalities, and preparation of the final report. Milestones help track progress and ensure that the project remains aligned with academic deadlines.

Task	Month 1	Month 2	Month 3	Month 4
Requirements Analysis				
System Design				
UI/ UX Design				
Frontend Development				
Backend Development				
Database Implementation				
System Integration				
Testing and Debugging				
Documentation and Reporting				
Final Review and Submission				

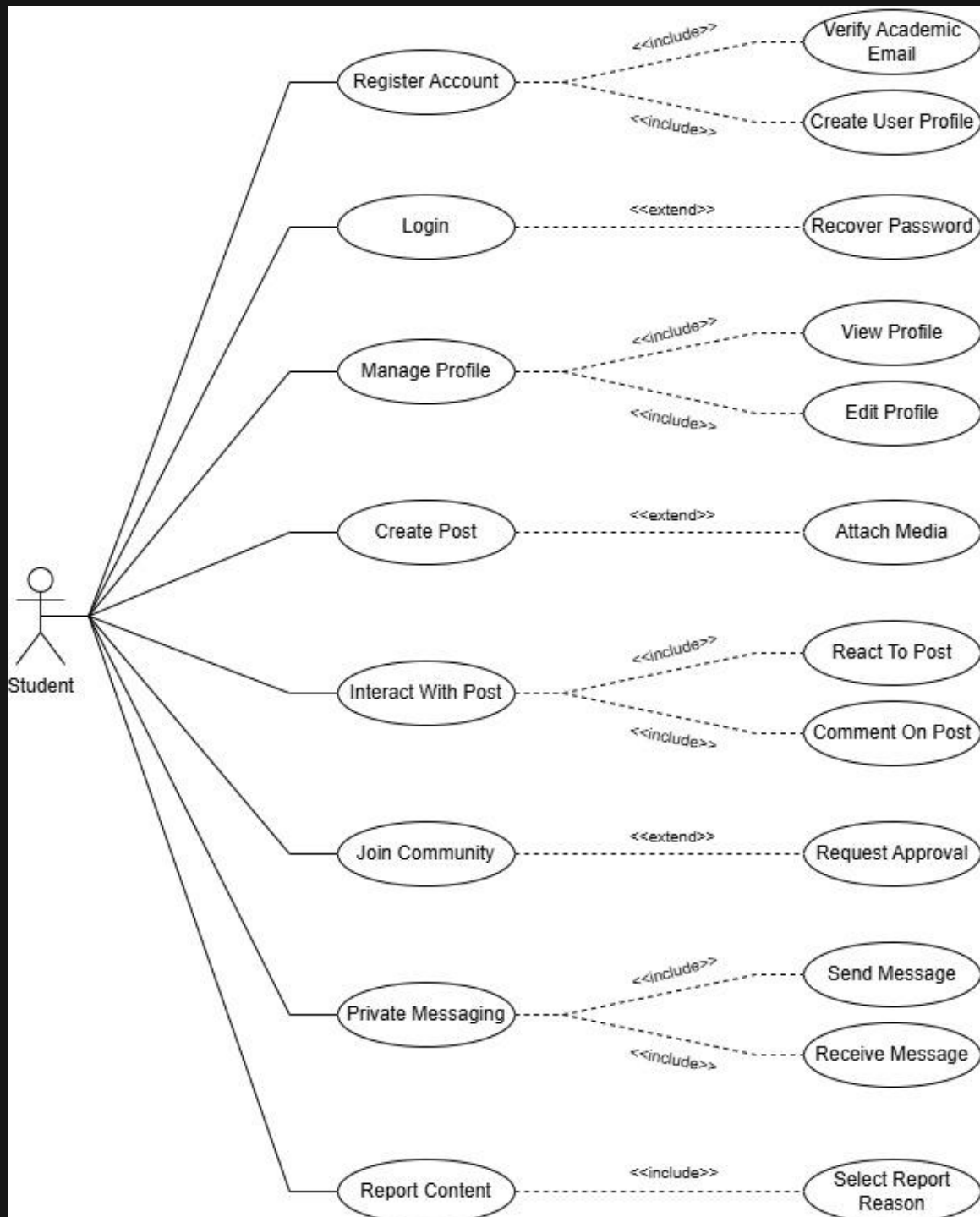
- **Risk Management:** Potential project risks include time constraints, technical challenges, and integration issues between system components. These risks are mitigated by adopting modular system design, prioritizing essential features, and maintaining flexibility in development planning. Regular testing and incremental integration reduce the likelihood of major system failures.
- **Quality Assurance:** Quality is maintained through continuous testing, code reviews, and validation of system requirements against implemented features. Functional testing ensures that system features operate as intended, while non-functional considerations such as usability and performance are reviewed throughout development.

Overall, effective project management ensures that **Project Campus** is developed in an organized, controlled, and adaptable manner, supporting successful completion within the available time and resources.

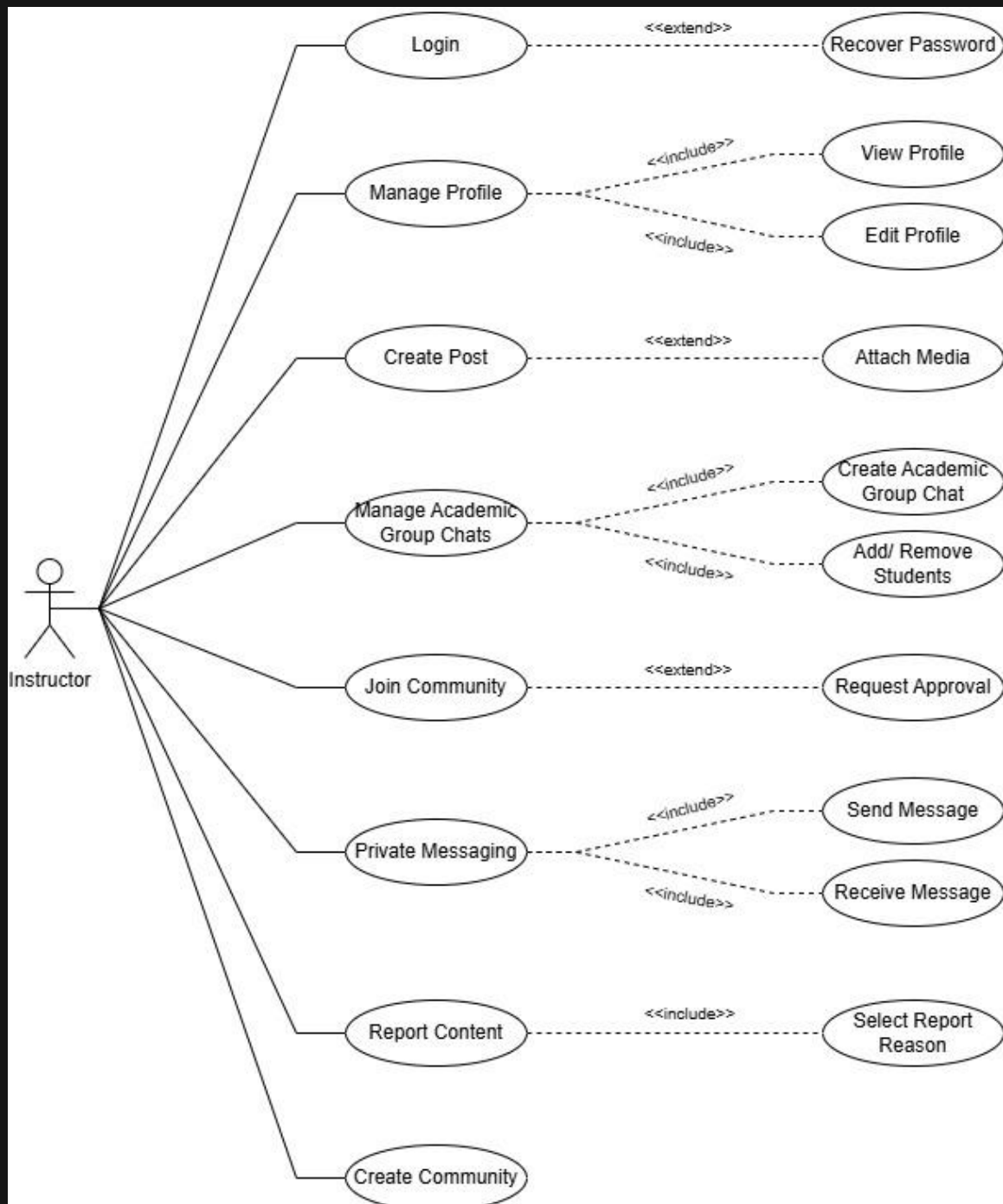
Diagrams

- **Use Case Diagram:** The use case diagrams illustrate the main interactions between system actors and the Project Campus platform. They define how Students, Instructors, and Pages (including University Pages) access system functionalities based on their roles and permissions. The diagrams emphasize role-based access control, showing shared behaviors such as authentication and content interaction, as well as specialized actions like academic group chat management by instructors and moderation and faculty management by university pages. The use of include and extend relationships clarifies mandatory and optional system behaviors, ensuring a clear and structured representation of system functionality.

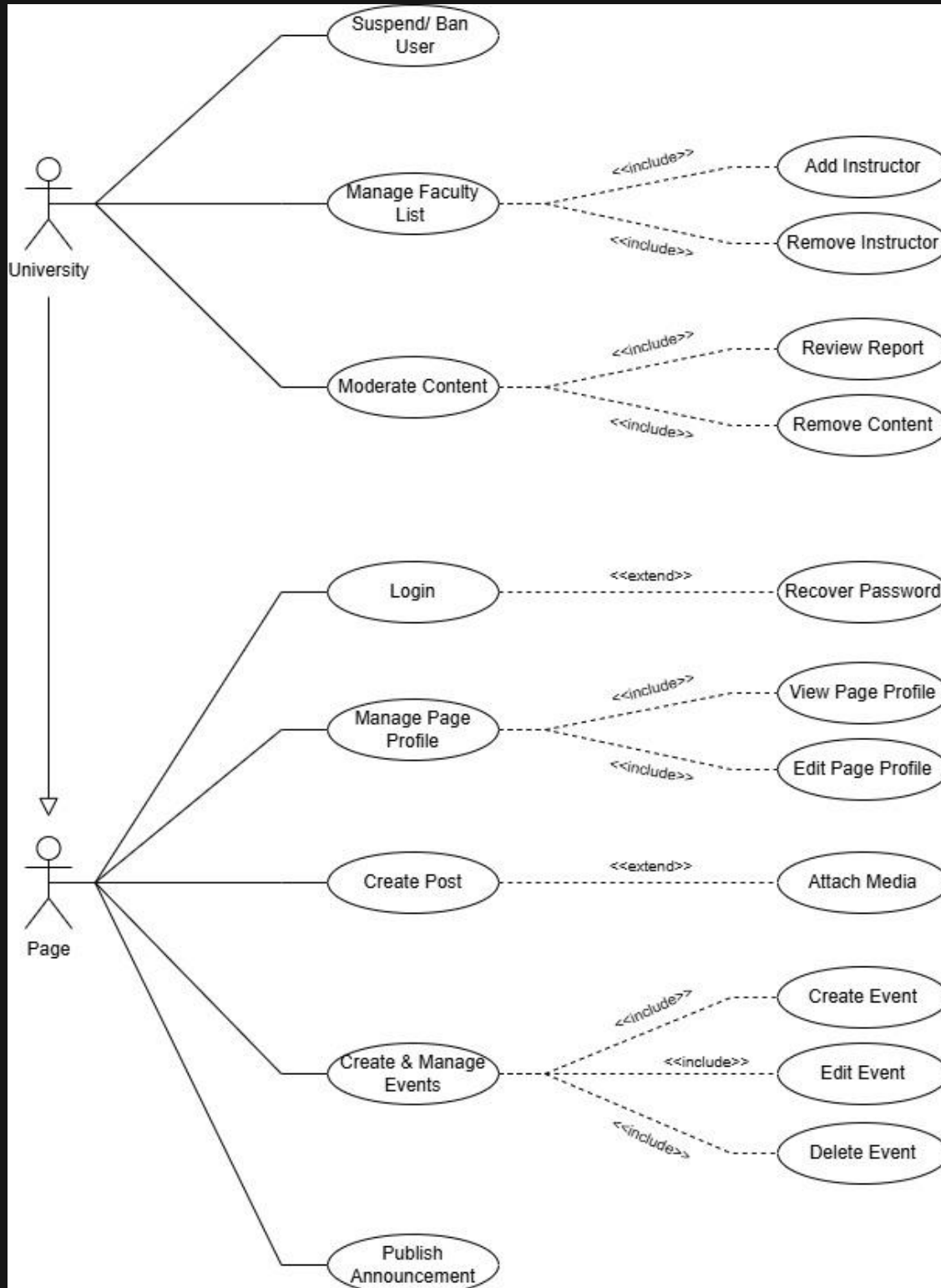
Student: Represents enrolled students who interact with the platform by creating and engaging with posts, joining communities, messaging other users, viewing announcements, and reporting inappropriate content. Students manage their personal profiles and participate in academic and social activities without administrative privileges.



Instructor: Represents academic staff members who publish academic content and manage course-related group chats in addition to general platform interactions such as messaging and community participation. Instructors play a leadership role in structured academic communication.

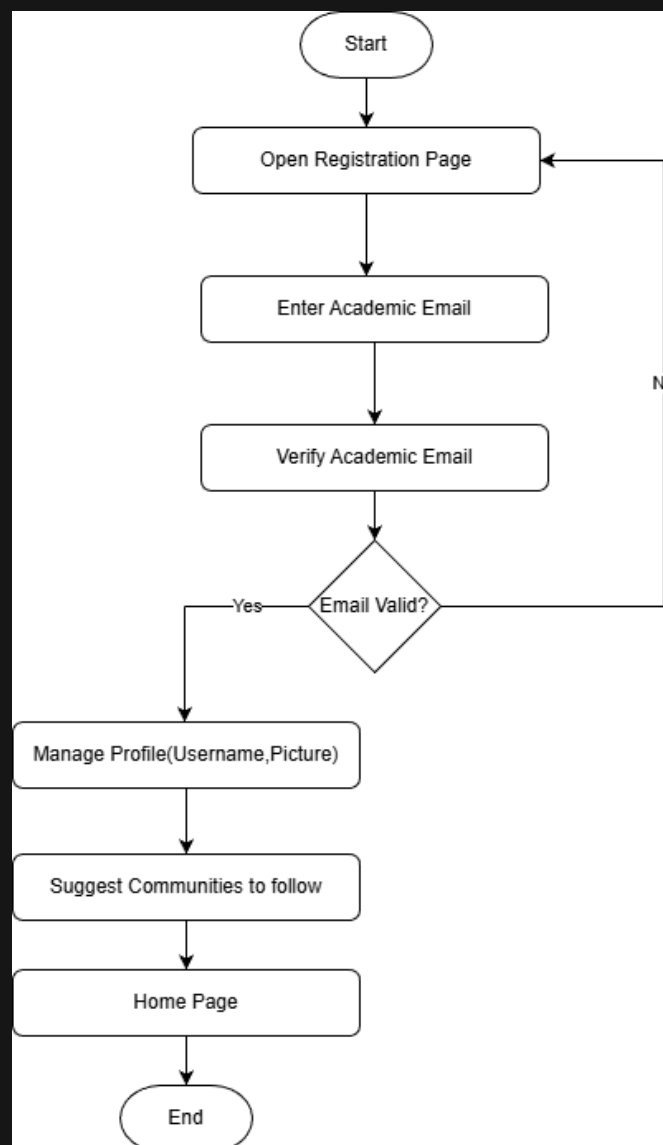


Page: Represents institutional or organizational pages that publish content, manage events, and maintain official page information. Pages interact with users through announcements and posts but do not have administrative authority.

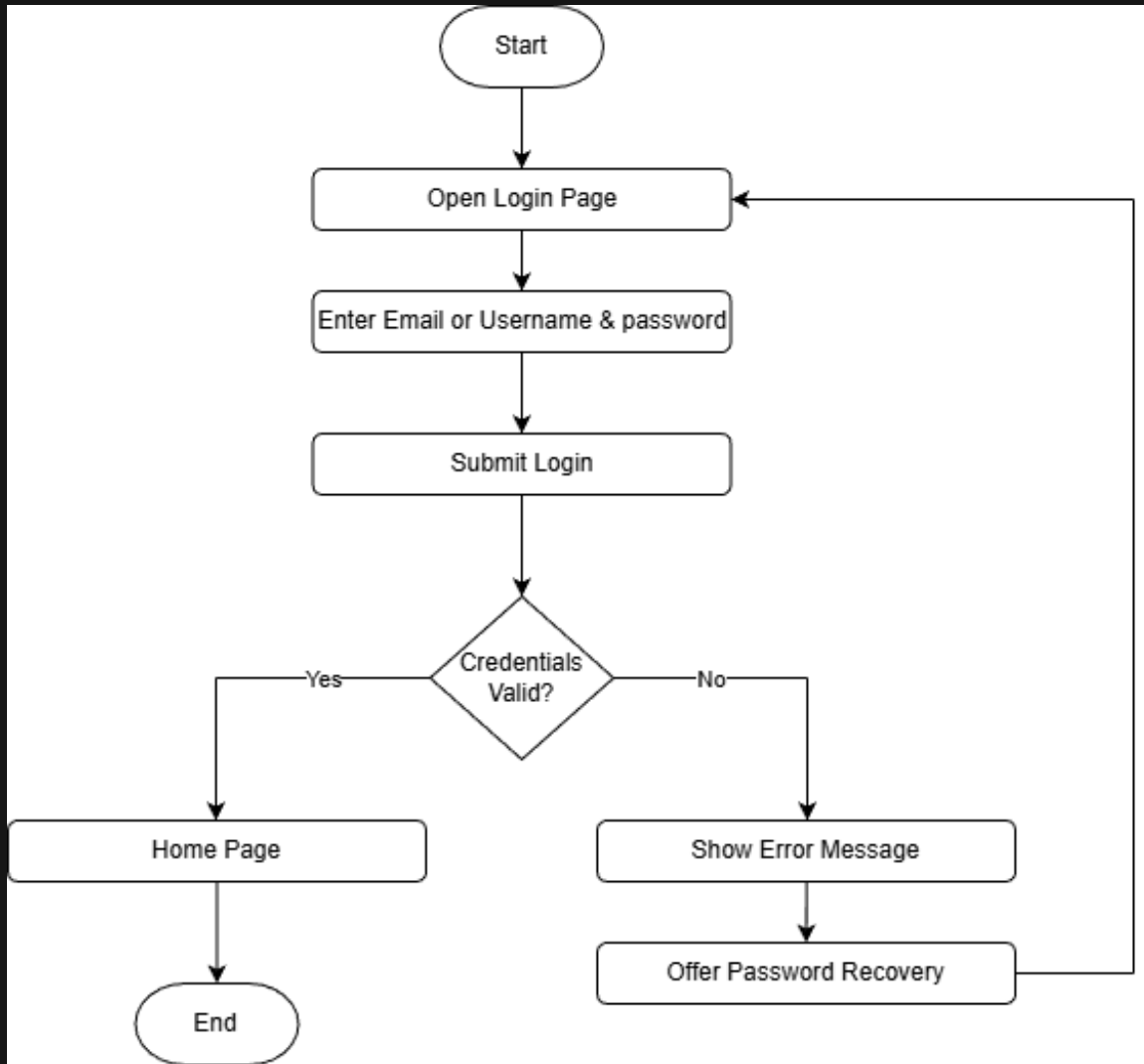


- **Flowchart Diagram:** The flowcharts illustrate the logical flow of key user and system processes in Project Campus. They present the sequence of actions and decision points involved in core functionalities, helping clarify how the system responds to different user interactions in a structured and easy-to-understand manner.

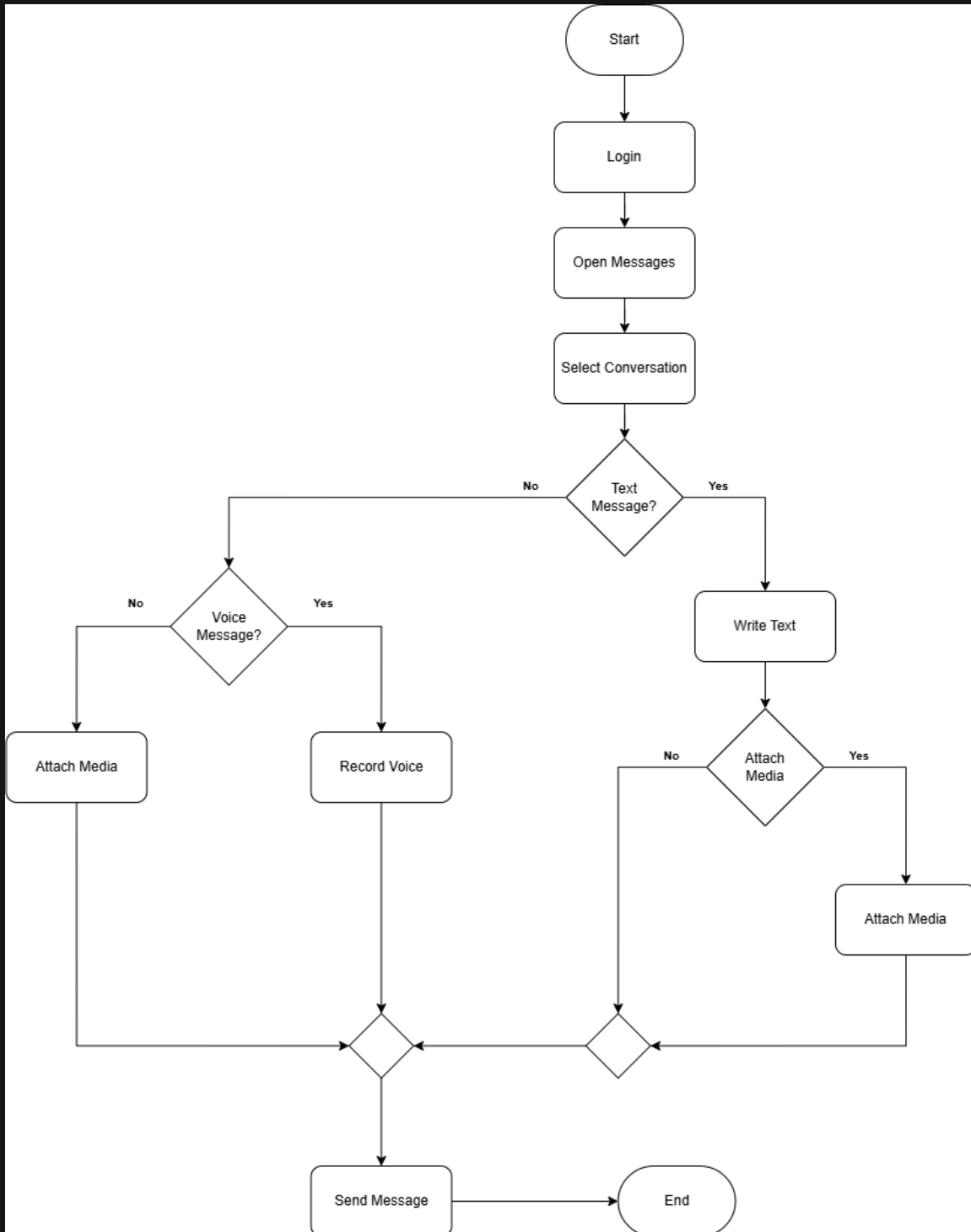
User Registration: describes the process of user registration, starting from opening the registration page to verifying the academic email address. It shows how valid users complete profile setup, receive community suggestions, and are directed to the home page.



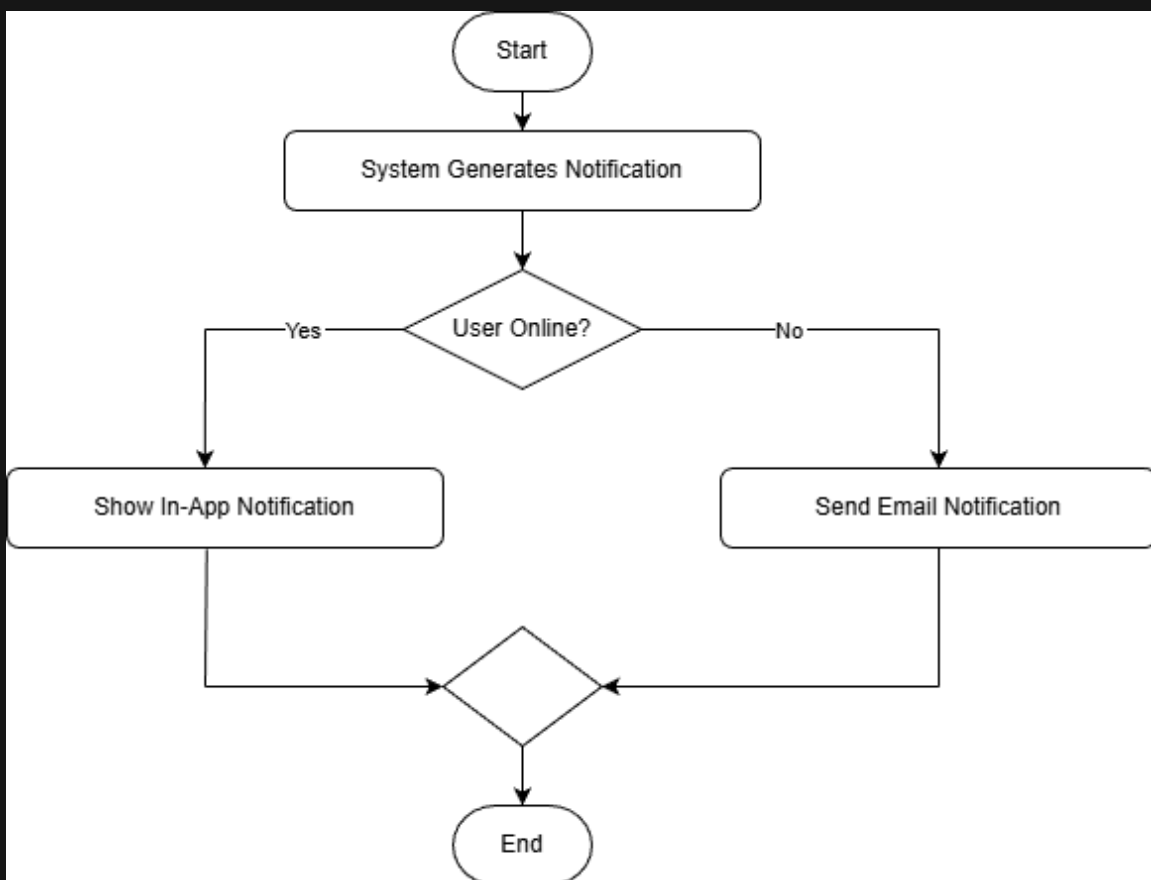
User Login: illustrates the login process, including credential submission and validation. It also handles invalid login attempts by displaying error messages and offering password recovery options.



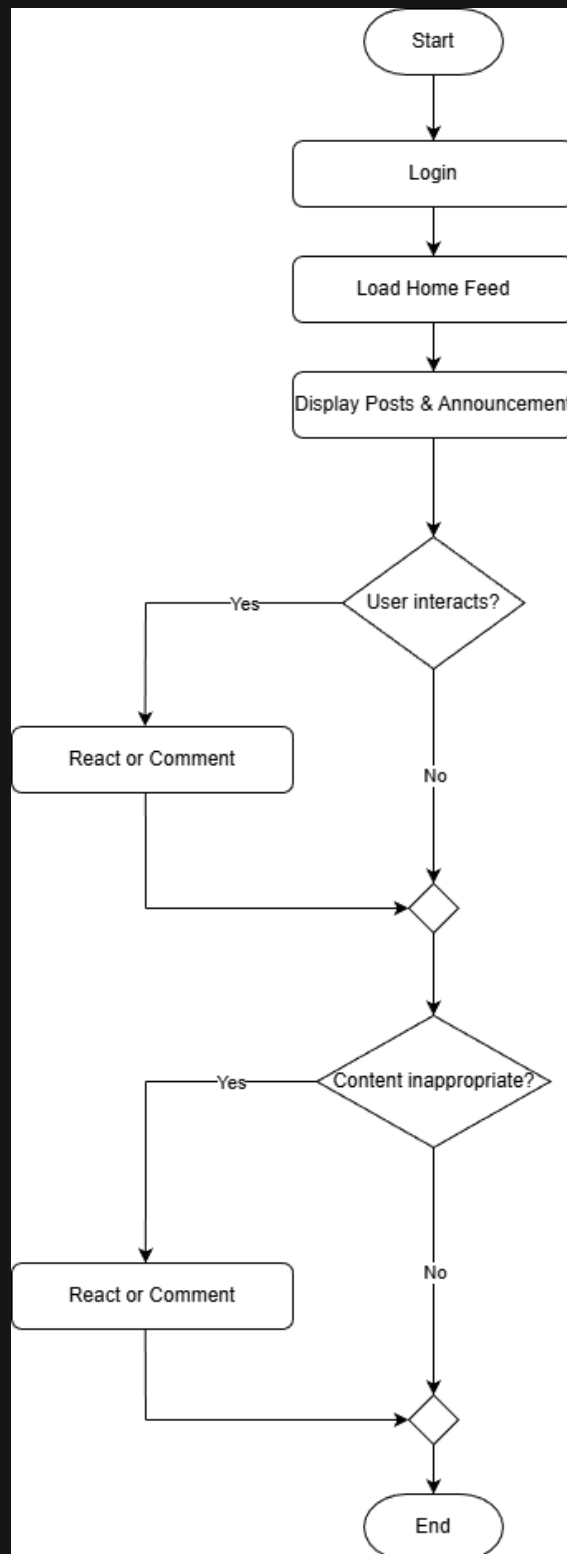
Send Message: represents the steps involved in sending messages within direct conversations. It supports text messages, media attachments, and voice messages, ensuring that all message types are processed correctly before being sent.



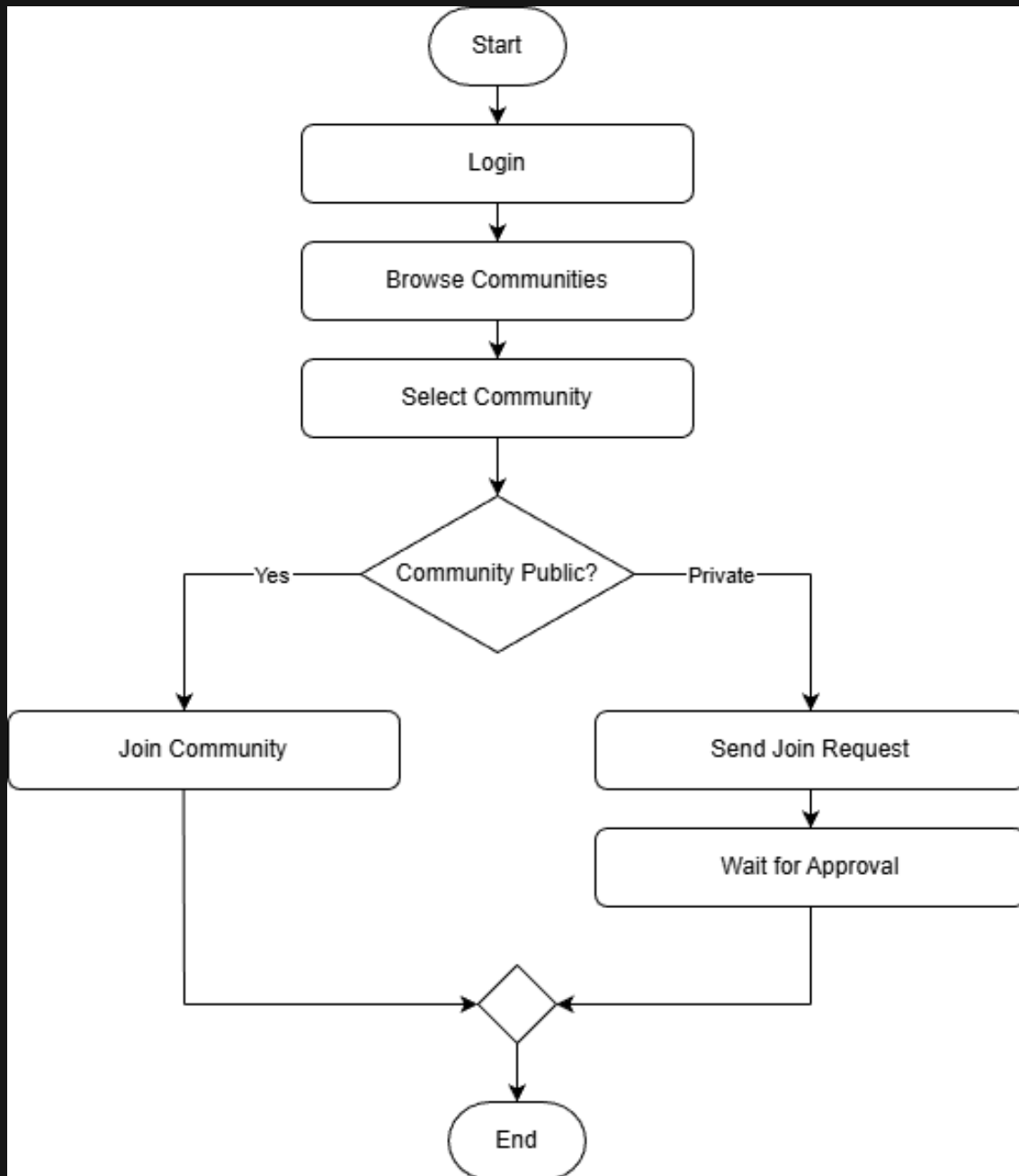
Notification Handling: explains how the system delivers notifications based on user availability. It shows that in-app notifications are displayed when users are online, while email notifications are sent when users are offline.



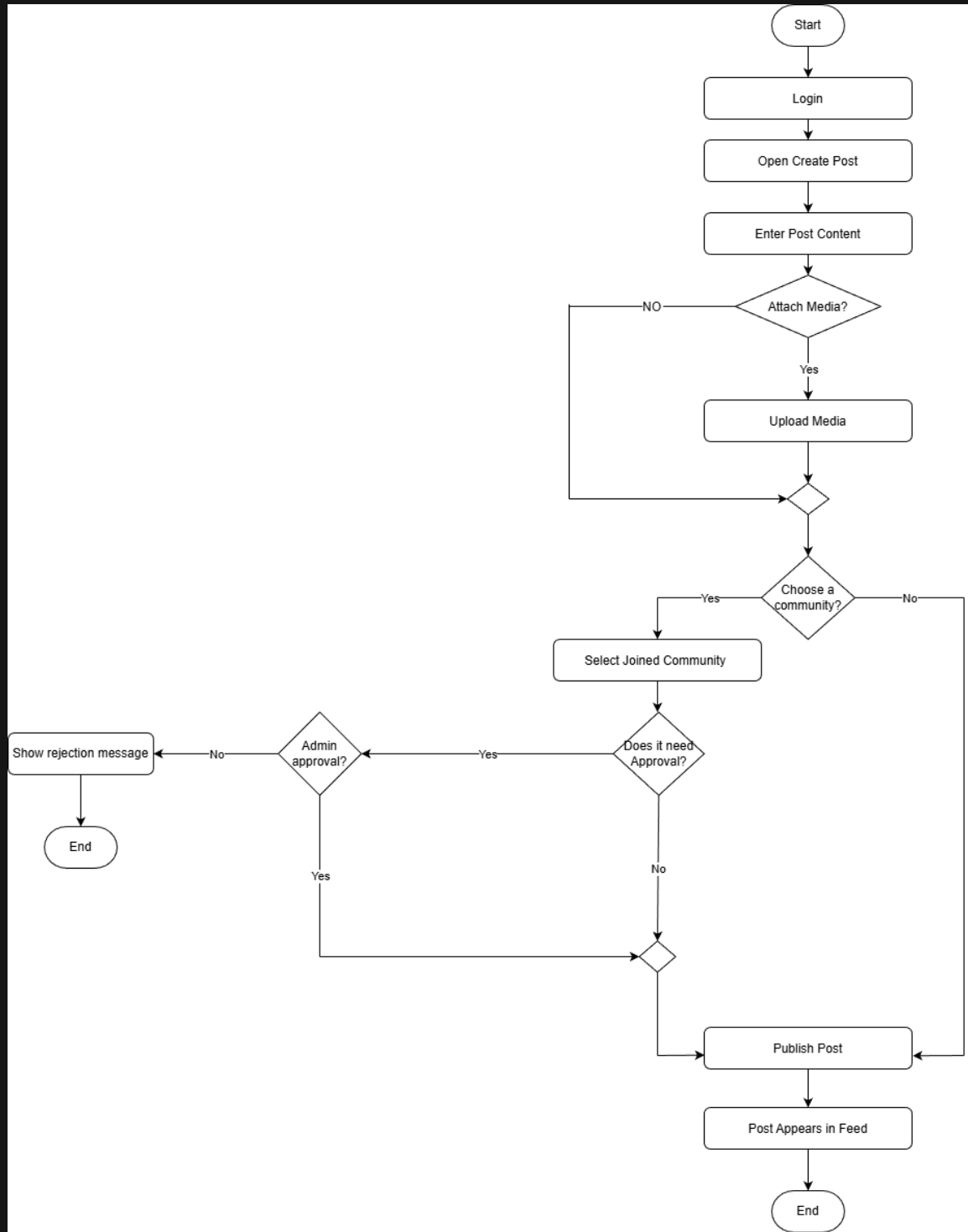
Interact with Post: outlines how users interact with posts by reacting or commenting. It also considers content appropriateness, ensuring that interactions follow platform guidelines.



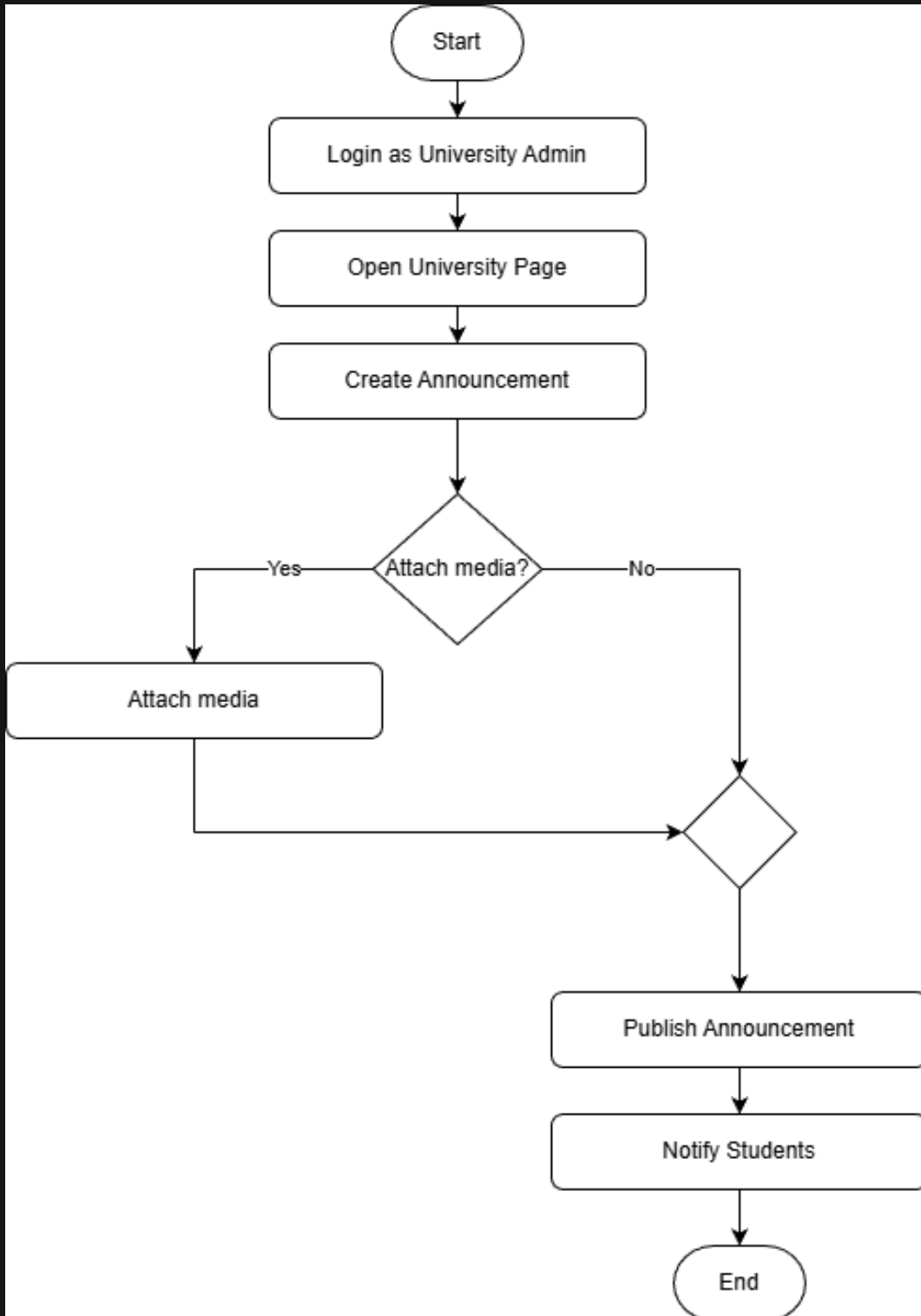
Join Community: illustrates the process of joining communities, distinguishing between public and private communities. It shows immediate access for public communities and approval-based access for private ones.



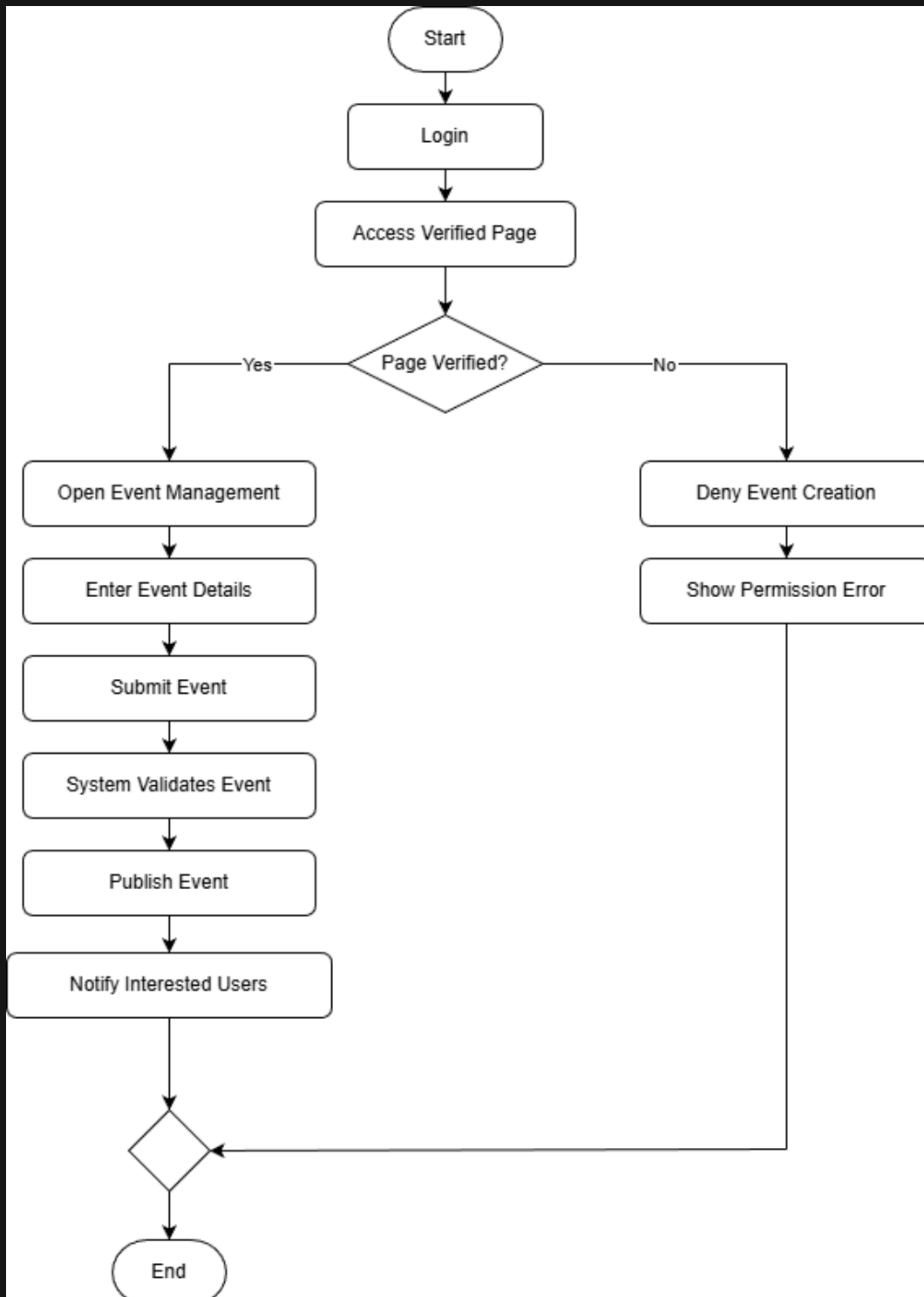
Create Post: describes the steps for creating a post, including entering content, attaching media, selecting a community, and handling approval requirements before publishing the post to the feed.



Create Announcement: explains how university administrators create and publish announcements. It includes optional media attachment and automatic notification of students after publication.

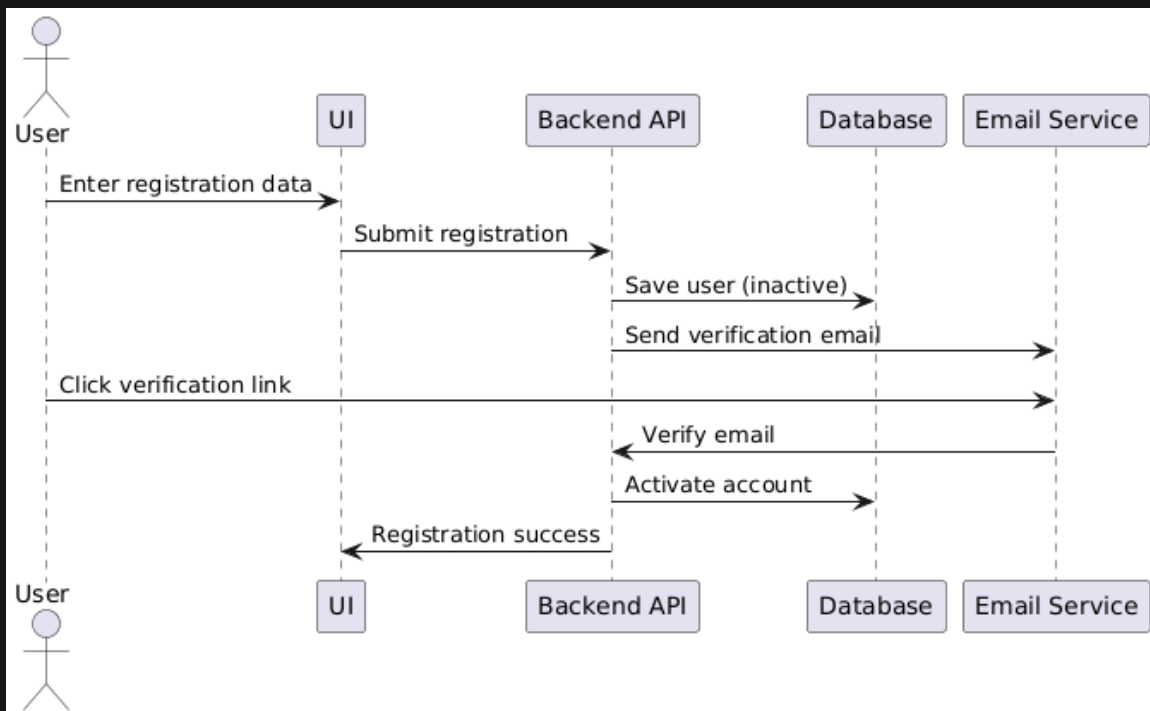


Create Event: represents the event creation process for verified pages. It shows permission validation, event submission, system verification, publication, and notification of interested users.

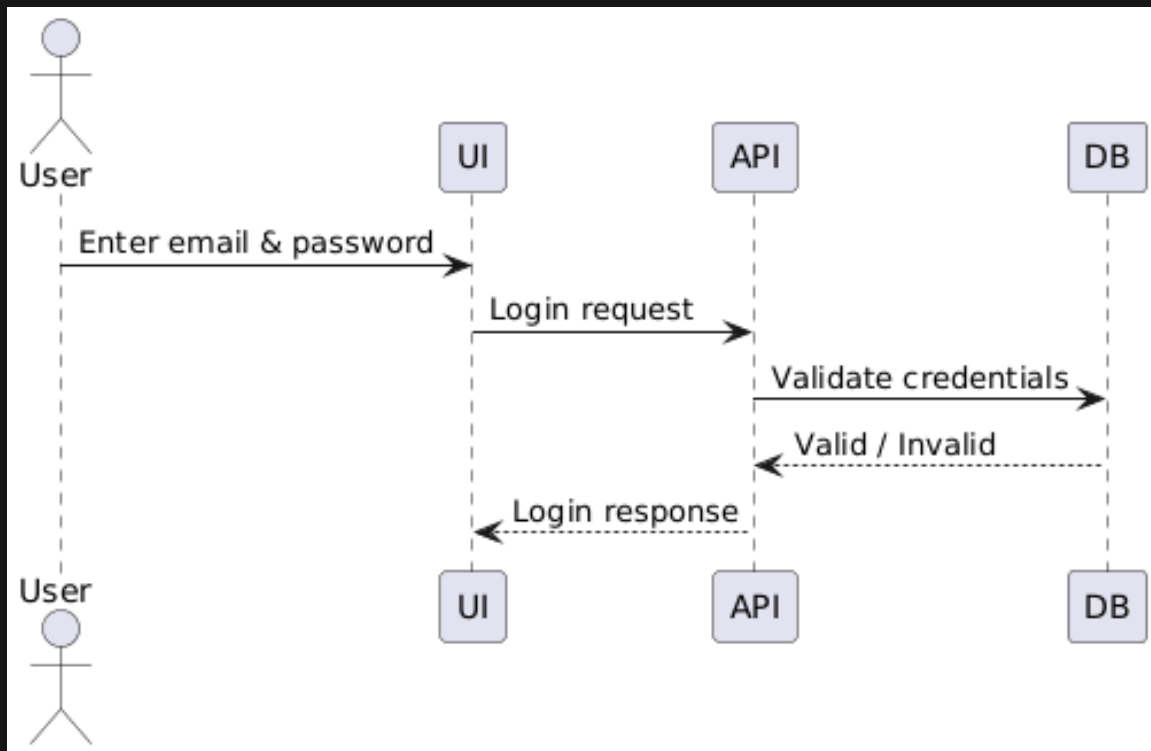


- **Sequence Diagram:** Sequence diagrams describe how actors and system components interact over time, illustrating the order of messages exchanged to complete specific system operations.

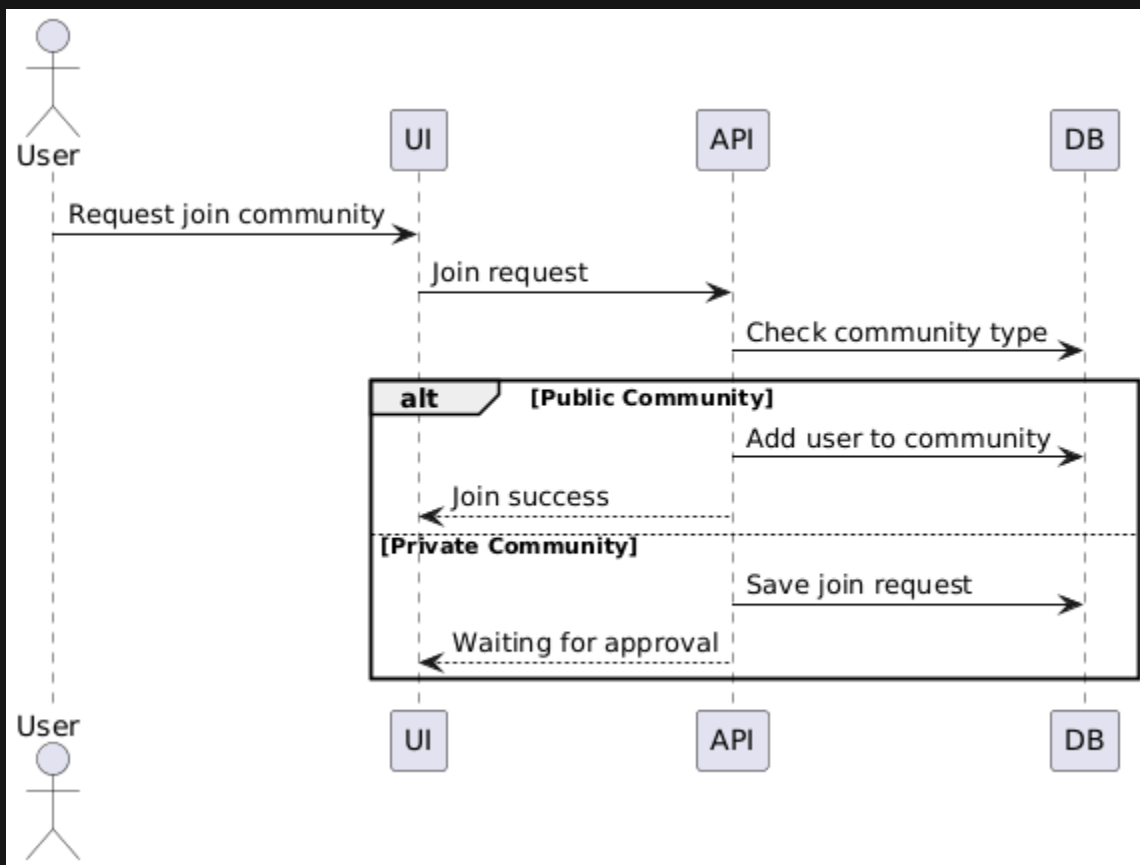
User Registration: This sequence diagram illustrates the registration process, where the user submits registration data through the interface and the backend creates an inactive account and sends a verification email. After email confirmation, the system validates the account and completes the registration.



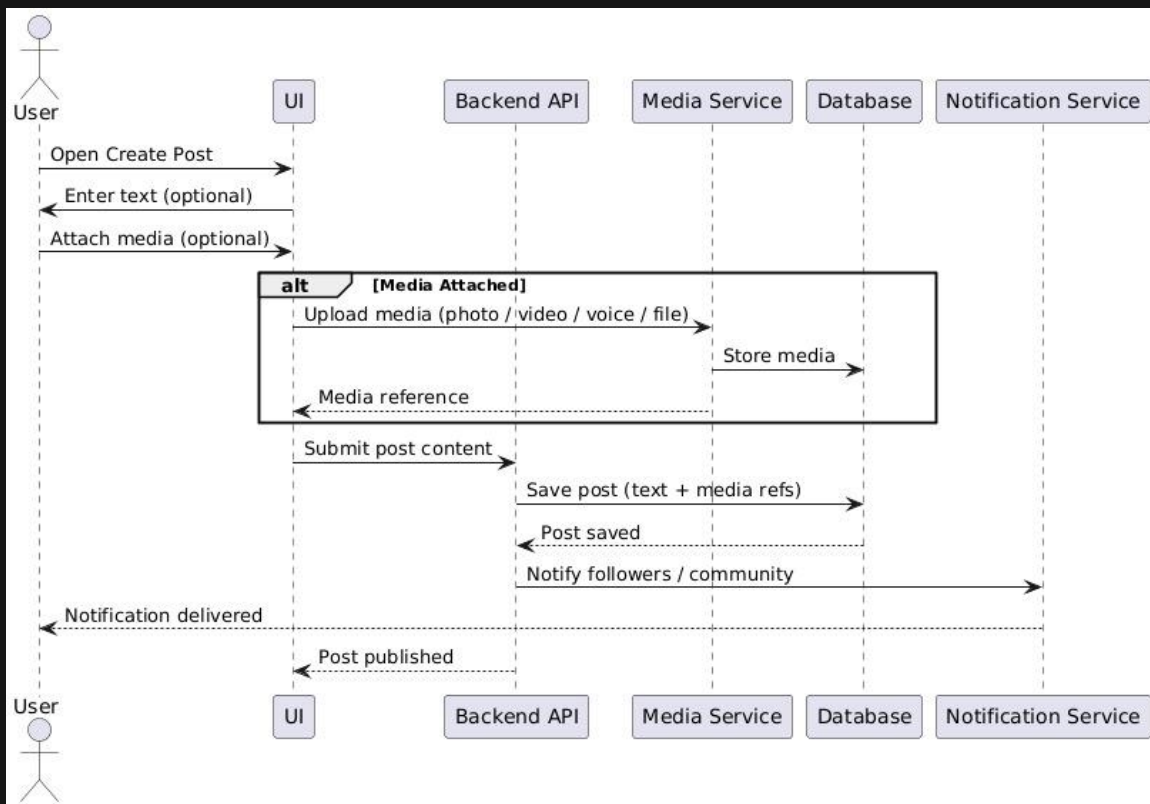
User Login: This sequence diagram shows the login process, where the user submits credentials through the interface and the backend validates them against stored data before granting or denying access.



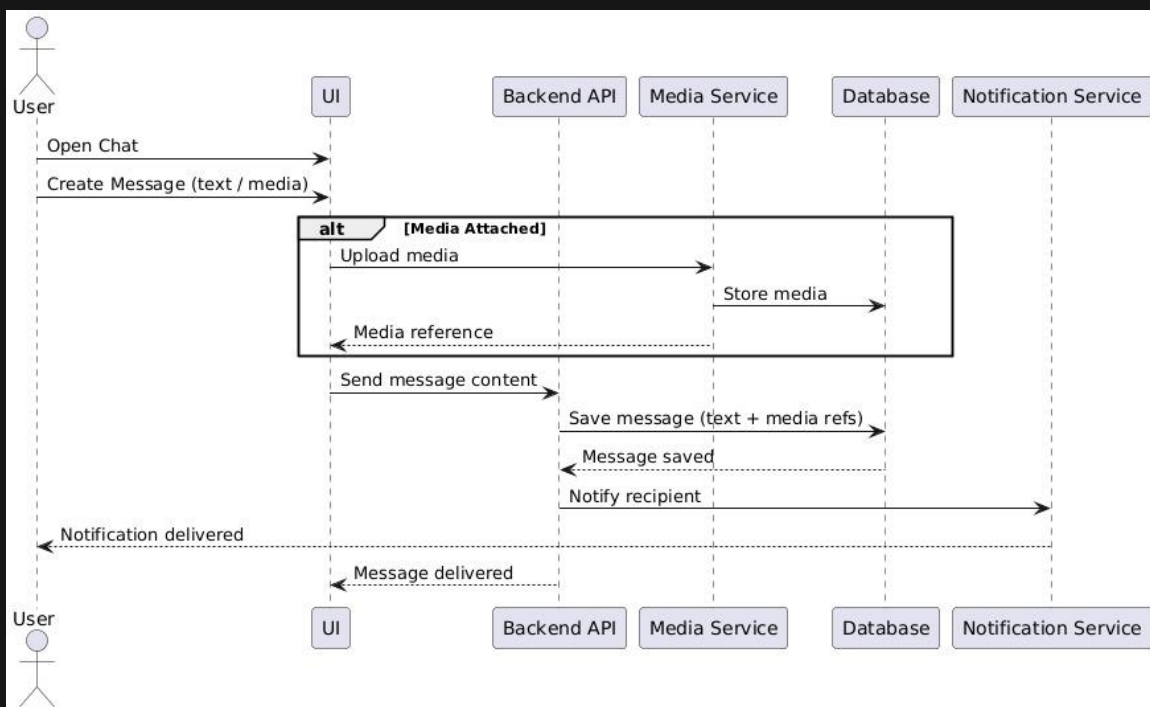
Join Community: This sequence diagram illustrates how a user joins a community by submitting a request. The system determines whether the community is public or private, granting immediate access for public communities or placing the user in a pending state until approval for private communities.



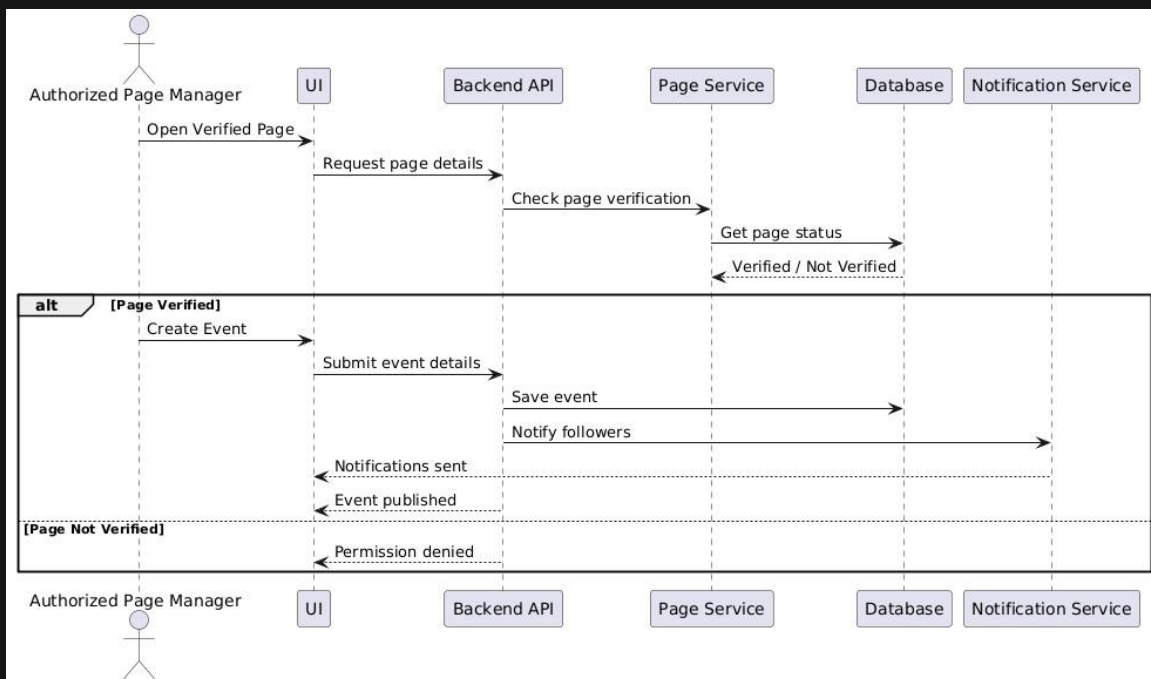
Create Post: This diagram illustrates the post creation process. The user enters text and optionally attaches media files. If media is attached, the content is uploaded to the media service and stored separately. The backend API then saves the post along with media references in the database. Once published, notifications are sent to relevant users or communities.



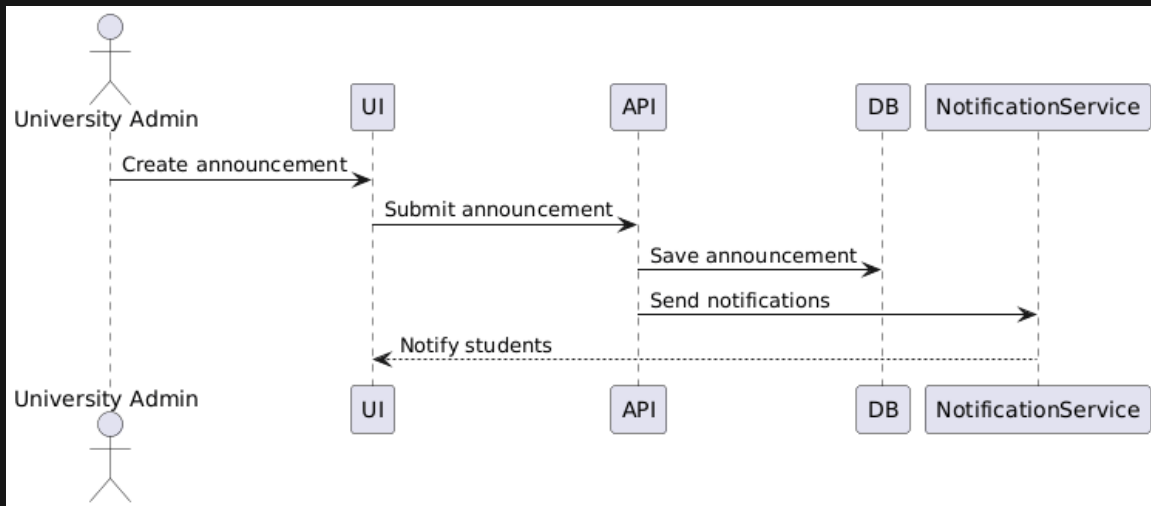
Send Message: This sequence diagram shows the messaging workflow. The user composes a message that may include text, media, voice recordings, or files. Attached media is processed by the media service before the message is saved in the database. After successful validation, the message is delivered to the recipient, and the notification service informs the receiving user.



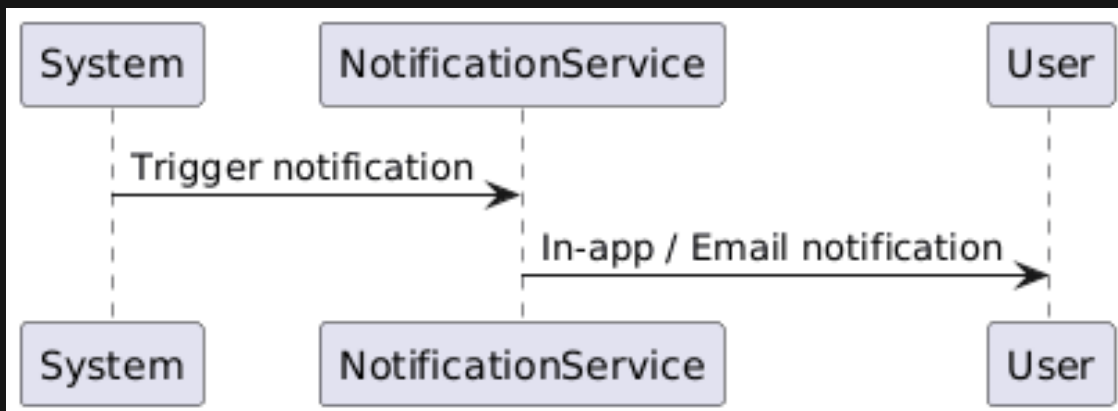
Create Event: This diagram represents the event creation process by an authorized page manager. The system verifies the page status before allowing event creation. If the page is verified, event details are submitted, stored in the database, and published. The system then notifies interested users. If the page is not verified, event creation is denied.



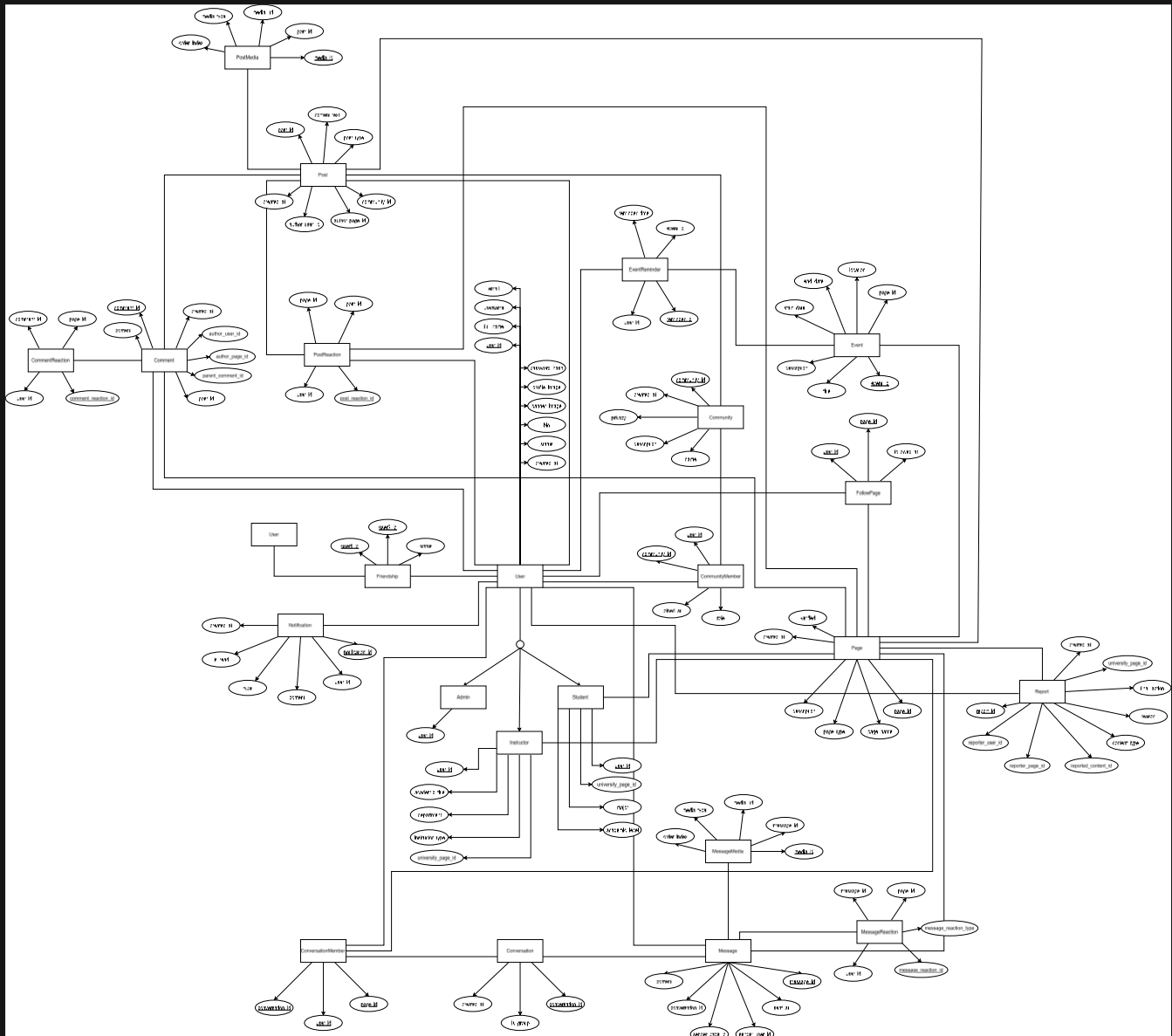
Publish Announcement (University Administrator): This sequence diagram illustrates the announcement publishing process. The university administrator submits an announcement through the interface, which is stored by the backend system. After successful storage, notifications are sent to students associated with the institution.



Receive Notification: This diagram shows how notifications are delivered. When an event occurs, the system triggers the notification service. If the user is online, an in-app notification is displayed. Otherwise, the system sends an email notification to ensure delivery.



- Entity Relationship Diagram:** The Entity Relationship Diagram illustrates the logical structure of the Project Campus database. It defines the main entities in the system, their attributes, and the relationships between them. The ERD was designed following database normalization principles to ensure data integrity, scalability, and efficient data access.

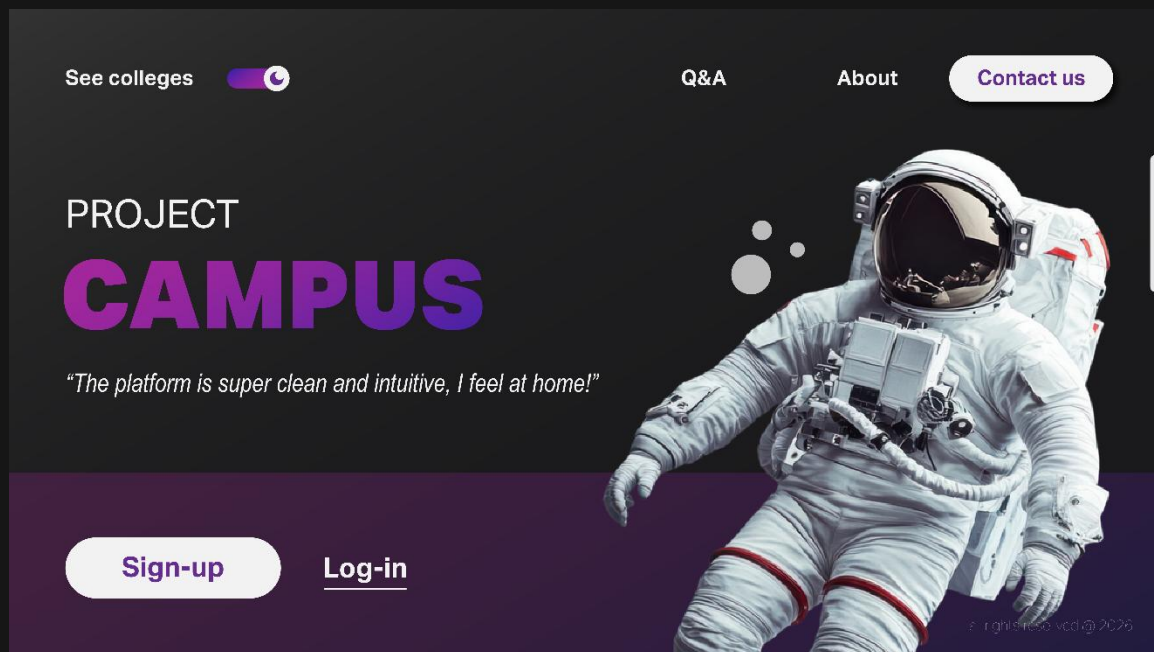


<i>Relationship</i>	<i>Type</i>	<i>Description</i>
User – Student / Instructor / Admin	Specialization (ISA)	User is specialized into roles to store role-specific attributes.
User – Friendship – User	Many-to-Many (Self)	Represents mutual friendships between users.
User – Community	Many-to-Many	Users can join multiple communities.
User – Conversation	Many-to-Many	Users can participate in multiple conversations.
User – Post	One-to-Many	A user can create multiple posts.
User – Message	One-to-Many	A user can send multiple messages.
Post – PostReaction – User	Many-to-Many	Users can react to posts.
Message – MessageReaction – User	Many-to-Many	Users can react to messages.
Post – PostMedia	One-to-Many	A post can contain multiple media items.
Message – MessageMedia	One-to-Many	A message can contain multiple media items.
Post – Comment	One-to-Many	A post can have multiple comments.
Comment – Comment	One-to-Many (Self)	Supports replies to comments.
Page – Post	One-to-Many	Pages can create multiple posts.
Page – Event	One-to-Many	Pages can create multiple events.

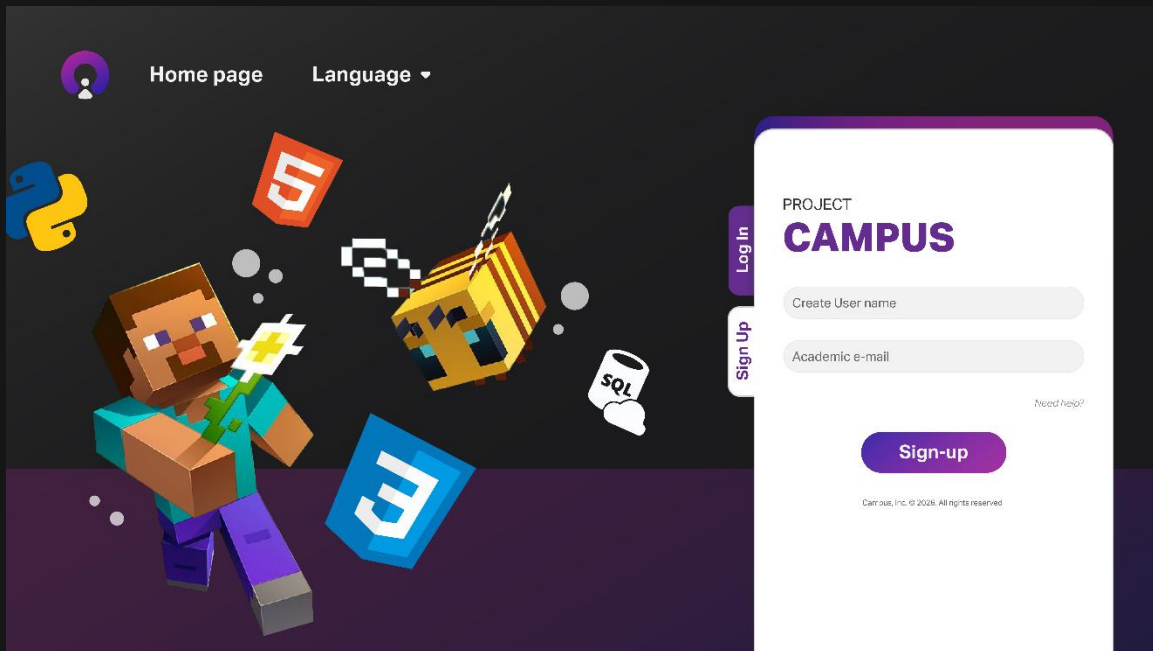
User Interface

The user interface of **Project Campus** is designed to provide a clean, intuitive, and consistent experience for all users, including students, instructors, and university administrators. The interface prioritizes ease of navigation, clarity of information, and accessibility to support efficient interaction within the academic environment.

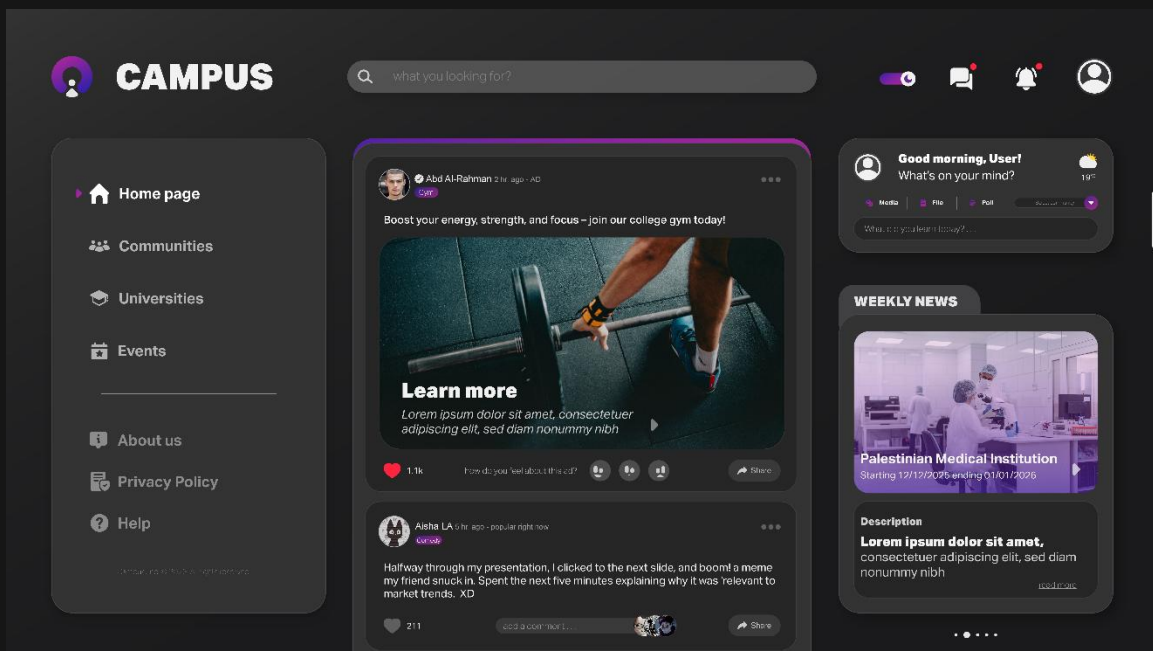
Landing Page: The landing page serves as the entry point to the platform and introduces users to the purpose and key features of Project Campus. It provides a brief overview of the platform's role in centralizing academic communication and highlights the benefits of using a unified university-focused system. Clear navigation options are provided to guide users toward authentication or further exploration of the platform.



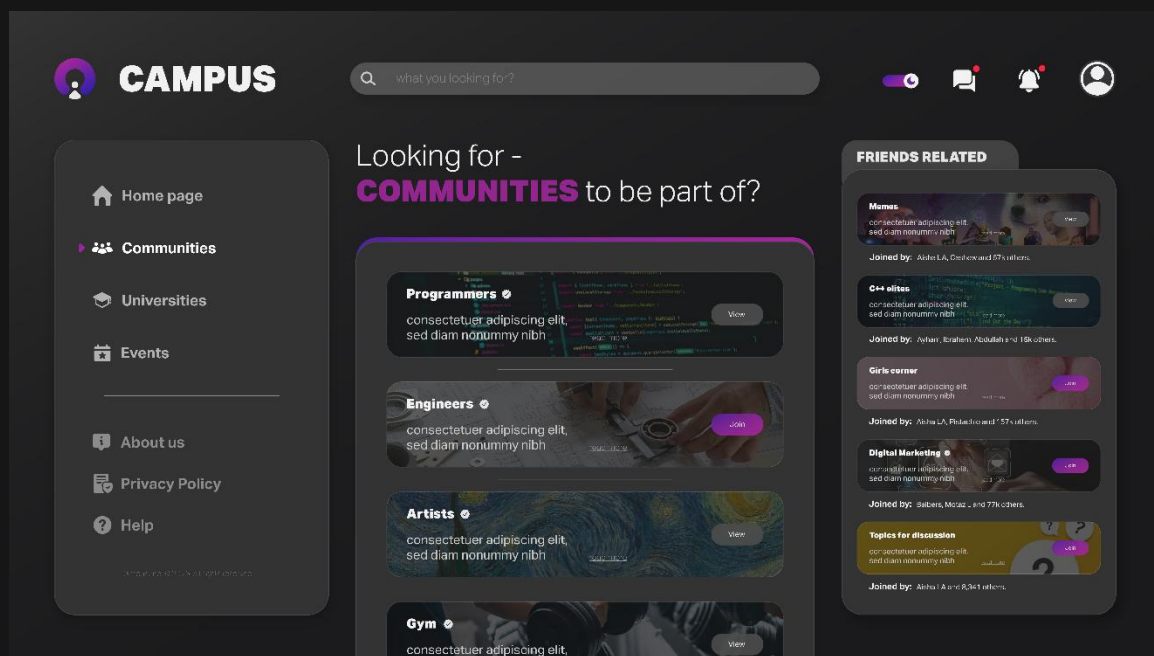
Login and Registration Page: The login and registration interface enables users to securely access the platform using their academic credentials. The design emphasizes simplicity and security, requiring users to register using official academic email addresses. The interface also supports password recovery and provides clear feedback to users during authentication processes.



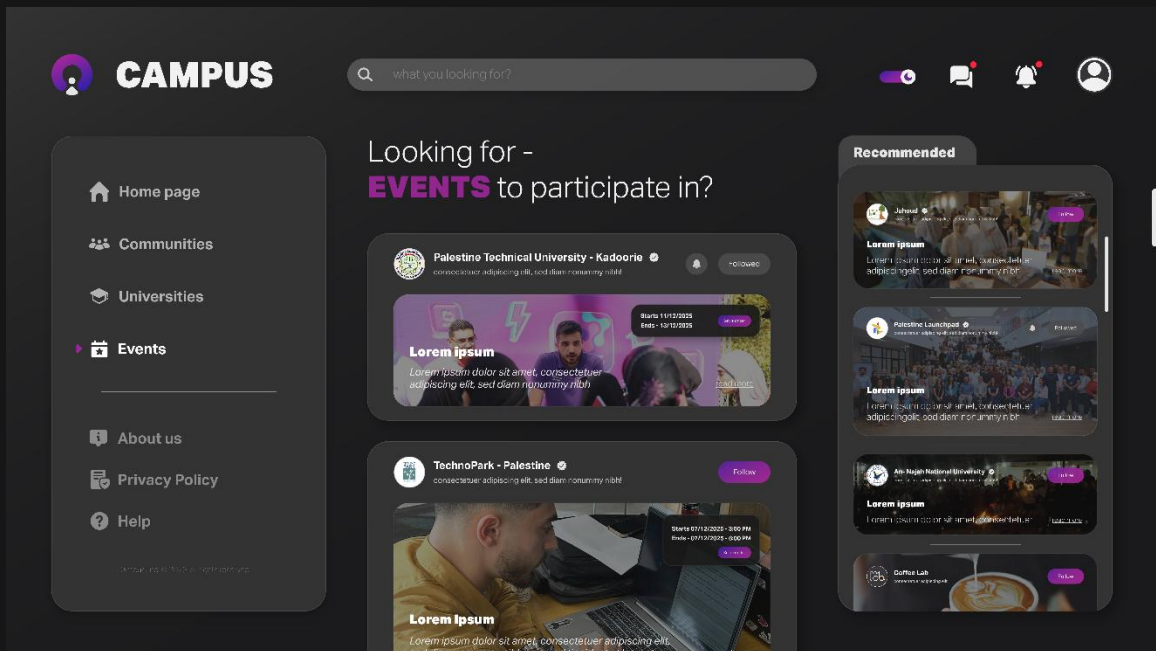
Home Page: The home page acts as the central hub of the platform, displaying a personalized feed of posts, official announcements, events, and community updates relevant to the user's university and interests. Users can create content, interact with posts, and receive notifications directly from this page. The layout is designed to balance information visibility with usability, ensuring that important academic updates are easily accessible.



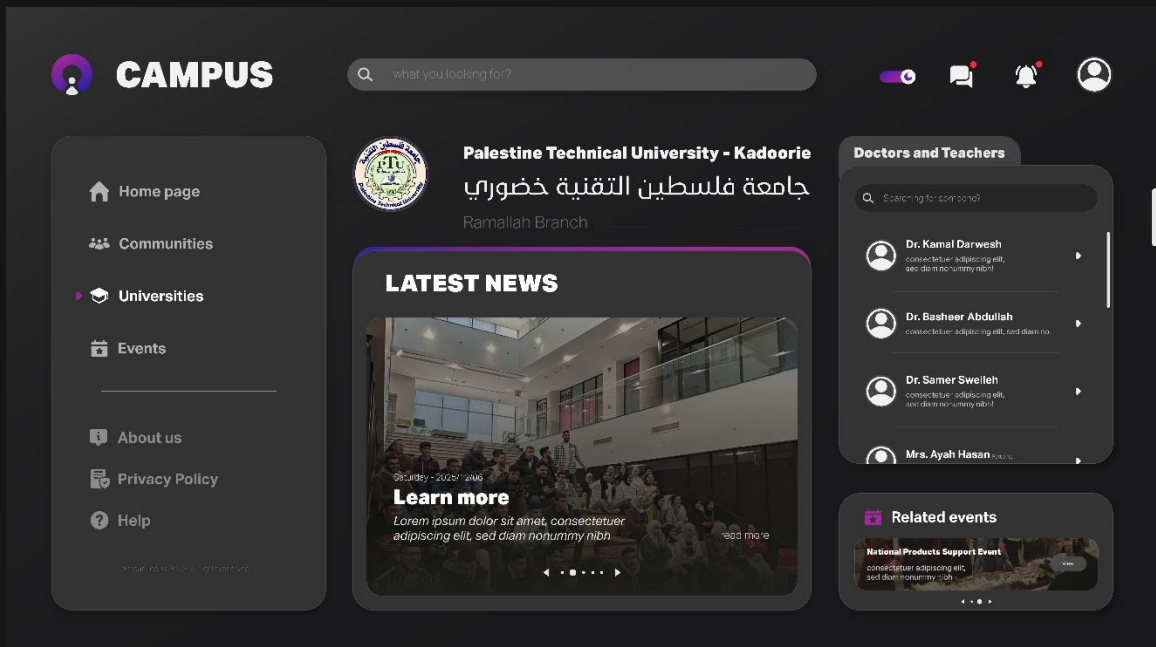
Communities Page: The communities page allows users to browse and join academic and interest-based communities. Communities may be public or private, depending on their configuration. Within each community, users can participate in discussions, share content, and engage with peers in a structured environment that supports collaboration and interaction.



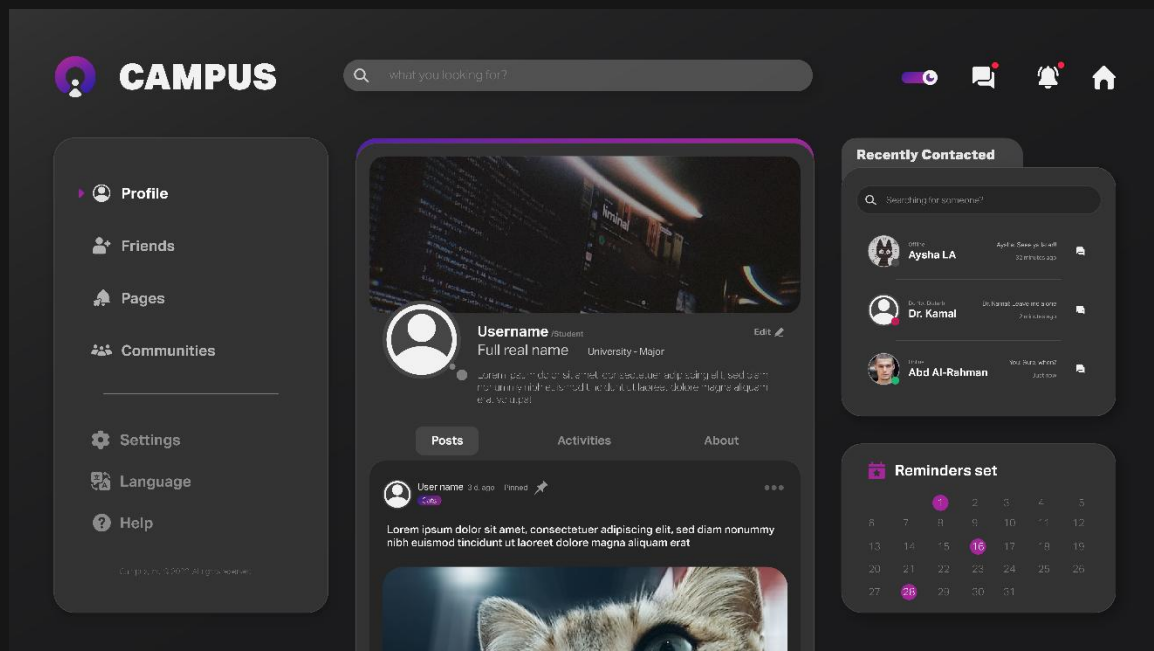
Events Page: The events page provides a centralized view of academic and social events published by authorized entities. Users can view event details, including date, time, location, and description, and choose to follow events to receive updates and reminders. This interface supports student engagement by connecting users with relevant campus activities.



University Page: The university page represents the official digital presence of an academic institution within the platform. It displays verified announcements, news, events, and faculty-related information published by authorized administrators. Users can follow university pages to receive official updates, ensuring reliable access to institutional information.



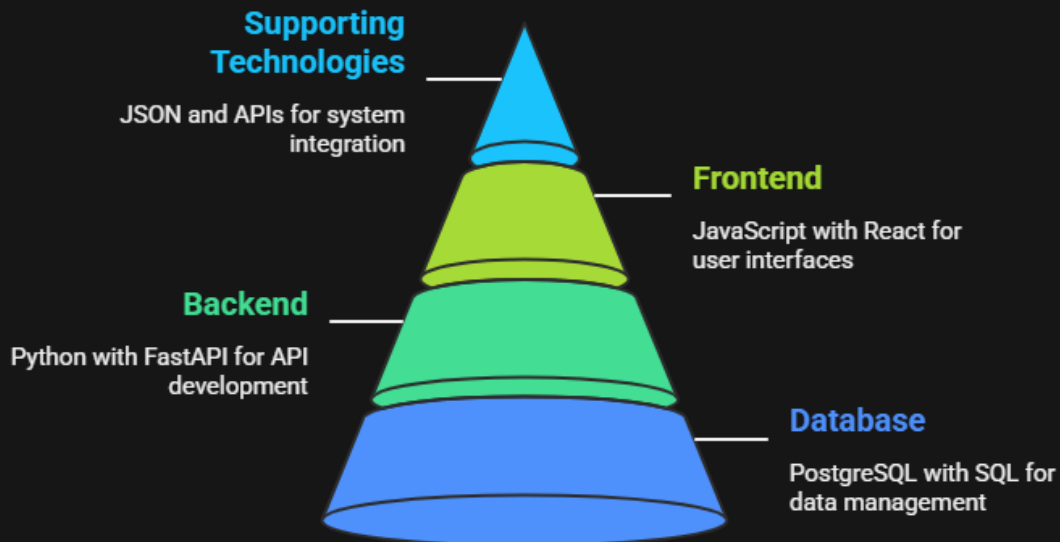
Profile Page: The profile page represents the personal digital identity of each user within the platform. It displays essential information such as the user's name, university, academic major, and a short bio, along with their profile picture and banner. The page provides access to the user's posts, activities, and general information, allowing others to understand their academic interests and engagement. Users can manage and update their profile details, ensuring accurate representation while maintaining a clear and organized personal presence within the campus community.



Design Consistency and Accessibility: The overall interface design maintains consistent visual elements, color schemes, and navigation patterns across all pages to enhance usability and reduce learning effort. The interface is responsive and adaptable to different screen sizes, ensuring accessibility across various devices. Language switching functionality supports both English and Arabic, allowing users to interact with the platform in their preferred language.

Coding Programming Languages

The development of **Project Campus** relies on a set of modern programming languages selected to support performance, scalability, and maintainability across different system layers.



Made with  Napkin

Overall, the selected programming languages and technologies provide a balanced development environment that supports the functional and non-functional requirements of Project Campus while allowing future expansion and system enhancements.

Conclusion

This project presented the design and development of **Project Campus**, a unified digital platform aimed at improving academic communication and engagement within university environments. The system was proposed to address the limitations of fragmented and informal communication tools by providing a structured, secure, and institution-centered solution tailored to students, instructors, and academic institutions.

Throughout the project, clear system requirements were identified, covering both functional and non-functional aspects to ensure reliability, usability, security, and scalability. The feasibility study demonstrated that Project Campus is economically, technically, and legally viable, supported by the use of open-source technologies, a scalable system architecture, and compliance with academic policies and privacy considerations.

The platform's design emphasizes verified identities, role-based access control, and institutional accountability, creating a safer and more reliable environment for academic interaction. Features such as university pages, communities, events, messaging, and multilingual support contribute to enhanced collaboration, improved information dissemination, and stronger connections between academic and student life.

In conclusion, Project Campus provides a solid foundation for a modern academic communication ecosystem. While the current implementation focuses on a single-university deployment, the system is designed to support future expansion and feature enhancement. The project successfully demonstrates how structured digital platforms can enhance academic communication, foster engagement, and support institutional integrity within higher education environments.

Future Work

Although Project Campus provides a strong foundation for academic communication, several enhancements can be considered for future development. One potential improvement is expanding the platform to support **multiple universities**, enabling broader academic interaction and collaboration across institutions.

Future versions of the system may include a **dedicated mobile application** to improve accessibility and provide a more seamless experience for users who primarily rely on mobile devices. Additional performance optimizations and infrastructure upgrades can also be implemented to support higher user loads and improved system availability.

The platform may also explore the integration of **intelligent support features**, such as a lightweight AI-based assistant to help users navigate the system, discover relevant events or communities, or access academic information more efficiently. This feature would be considered an enhancement rather than a core system component.

In addition, a **simple reputation or social point mechanism** could be introduced to encourage positive participation and responsible interaction within the platform. Such a mechanism would aim to promote constructive engagement while maintaining the academic integrity of the system.

These future enhancements would allow Project Campus to evolve gradually while preserving its primary goal of providing a secure, structured, and institution-centered academic communication platform.